

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра №806 «Вычислительная математика и программирование»**

**Курсовой проект
по курсу «Основы криптографии»**

**Реализация алгоритмов шифрования DES, TripleDES, DEAL, RSA, Rijndael,
RC4, LOKI97, протокол Диффи-Хелмана**

Выполнил: Колесник Д.С.

Группа: 8О-313Б

Преподаватель: А.М. Романенков

Москва, 2025

Содержание

Содержание	2
Введение	6
Теоретическая часть.....	7
DES.....	7
История	7
Общая структура	7
Начальная и финальная перестановки	8
Раундовая функция Фейстеля.....	8
Раундовая функция f	8
Генерация раундовых ключей	9
Процесс расшифрования.....	10
Криптографическая стойкость.....	10
Математические свойства.....	11
TripleDES	11
Предпосылки создания и стандартизация.....	11
Принцип работы алгоритма	12
Варианты использования ключей.....	12
Альтернативная схема EEE	14
Криптоанализ и безопасность.....	14
Производительность	15
Режимы работы	15
DEAL.....	16
Происхождение и контекст разработки	16

Основная структура алгоритма	16
Раундовая функция	17
Генерация раундовых ключей	17
Количество раундов	18
Процесс шифрования и расшифрования.....	18
Криптографические атаки.....	18
Производительность	19
RSA	19
История создания и значение	19
Математические основы	20
Генерация ключей	20
Процесс шифрования	22
Процесс расшифровки	22
Оптимизация с китайской теоремой об остатках	23
Криптоаналитические атаки	23
Современное применение	24
Rijndael.....	24
История создания и конкурс AES	24
Основные параметры	25
Математическая основа	25
Структура алгоритма.....	25
Раундовые преобразования.....	26
Расширение ключа	28
Процесс шифрования	28
Процесс расшифрования.....	29

Криптобезопасность и атаки.....	29
RC4.....	30
История	30
Основные характеристики	31
Структура алгоритма.....	31
Математические свойства.....	32
Атаки.....	32
LOKI97.....	33
История	33
Основные параметры	33
Структура алгоритма.....	33
Раундовая функция	34
Расписание ключей	35
Криптостойкость	35
Протокол Диффи-Хеллмана	36
История	36
Математические основы	36
Классический Диффи-Хеллман.....	37
Выбор параметров.....	38
Атаки.....	39
Практическая часть	41
DES.....	41
TripleDES	51
DEAL.....	53
RSA	57

Rijndael.....	66
RC4.....	79
LOKI97.....	82
Diffie-Hellman.....	90
Вывод.....	93
Список использованных источников.....	94
Приложение А Репозиторий с исходным кодом.....	95

Введение

В современном цифровом мире проблема защиты информации приобретает критическое значение. Ежедневно через глобальные сети передаются терабайты конфиденциальных данных: банковские транзакции, персональные сведения, корпоративная информация, государственные секреты. Обеспечение конфиденциальности, целостности и аутентичности этих данных становится одной из ключевых задач информационной безопасности.

Криптографические алгоритмы составляют фундамент современных систем защиты информации. От классических симметричных шифров, таких как DES (Data Encryption Standard) и его модификаций, до современных стандартов вроде AES (Advanced Encryption Standard, базирующегося на алгоритме Rijndael), от асимметричных систем типа RSA до поточных шифров и протоколов обмена ключами — все эти технологии формируют многоуровневую систему защиты данных в различных сценариях применения.

Особую важность приобретает не только теоретическое понимание криптографических примитивов, но и практические навыки их реализации и применения. Разработка собственных реализаций криптографических алгоритмов позволяет глубже понять их внутренние механизмы, выявить потенциальные уязвимости и ограничения, а также научиться правильно интегрировать криптографические компоненты в прикладные системы.

Практическая значимость данной курсовой работы заключается в создании функционального программного комплекса, который может применяться как в образовательных, так и в исследовательских целях. Разработанная библиотека криптографических алгоритмов представляет собой модульную архитектуру с четко определенными интерфейсами, что позволяет легко интегрировать отдельные компоненты в сторонние проекты. Реализованный графический интерфейс на базе Avalonia UI обеспечивает кроссплатформенность решения и может использоваться для демонстрации работы криптографических алгоритмов в учебном процессе, позволяя студентам наглядно изучать процессы шифрования и расшифрования данных в различных режимах работы. Комплексный набор unit-тестов и stress-тестов, разработанный в рамках проекта, служит эталоном корректности реализации и может применяться для верификации других криптографических библиотек.

Теоретическая часть

Криптография как наука о методах обеспечения конфиденциальности и аутентичности информации прошла многовековой путь развития — от примитивных шифров подстановки древнего мира до математически обоснованных алгоритмов современности, стойкость которых базируется на вычислительной сложности определенных математических задач. Современные криптографические алгоритмы подразделяются на два основных класса: симметричные системы, где один и тот же ключ используется для шифрования и расшифрования, и асимметричные системы с парой математически связанных ключей — открытым и закрытым. В данной главе будут рассмотрены математические основы и принципы работы алгоритмов, реализованных в рамках курсовой работы: блочных симметричных шифров с различными режимами работы, асимметричного алгоритма RSA, поточного шифра RC4 и протокола обмена ключами Диффи-Хеллмана.

DES

История

Data Encryption Standard (DES) — симметричный блочный шифр, разработанный компанией IBM в начале 1970-х годов на основе проекта Lucifer и утвержденный Национальным институтом стандартов и технологий США (NIST) в качестве федерального стандарта обработки информации FIPS PUB 46 в 1977 году. DES стал первым общедоступным криптографическим алгоритмом, одобренным правительством для защиты секретной, но не совершенно секретной информации. На протяжении более чем 20 лет DES являлся доминирующим стандартом шифрования в коммерческих и государственных приложениях, пока в 2001 году не был заменен алгоритмом AES.

Общая структура

DES представляет собой блочный шифр с размером блока 64 бита и длиной ключа 56 бит (формально 64 бита, но 8 бит используются для контроля четности). Алгоритм основан на сети Фейстеля и выполняет 16 раундов преобразований над входным блоком данных. Основная идея сети Фейстеля заключается в разделении блока данных на две половины и поочередном их преобразовании с использованием раундовых ключей, что обеспечивает как рассеивание, так и перемешивание информации.

Общая схема работы DES включает три основных этапа:

1. Начальная перестановка (Initial Permutation, IP) — фиксированная перестановка битов входного блока
2. 16 раундов преобразования Фейстеля — итеративное применение раундовой функции
3. Финальная перестановка (Final Permutation, FP) — обратная начальной перестановка

Начальная и финальная перестановки

Начальная перестановка IP принимает 64-битный блок открытого текста и переставляет биты согласно фиксированной таблице перестановок. Биты переупорядочиваются таким образом, что бит с позиции i перемещается на позицию IP(i). Таблица начальной перестановки имеет следующий вид: IP = [58, 50, 42, 34, 26, 18, 10, 2, 60, 52, 44, 36, 28, 20, 12, 4, 62, 54, 46, 38, 30, 22, 14, 6, 64, 56, 48, 40, 32, 24, 16, 8, 57, 49, 41, 33, 25, 17, 9, 1, 59, 51, 43, 35, 27, 19, 11, 3, 61, 53, 45, 37, 29, 21, 13, 5, 63, 55, 47, 39, 31, 23, 15, 7].

Финальная перестановка FP является обратной операцией к IP, то есть $FP = IP^{-1}$, и применяется к результату 16-го раунда перед получением итогового шифртекста.

Раундовая функция Фейстеля

После начальной перестановки 64-битный блок разделяется на две 32-битные половины: левую L_0 и правую R_0 . Каждый из 16 раундов применяет следующее преобразование:

$$\begin{aligned} L_i &= R_{i-1} \\ R_i &= L_{i-1} \oplus f(R_{i-1}, K_i) \end{aligned}$$

где K_i — раундовый ключ для раунда i , $\oplus$ — операция побитового исключающего ИЛИ (XOR), а f — раундовая функция DES.

Раундовая функция f

Раундовая функция $f(R, K)$ является ключевым компонентом DES и состоит из четырех последовательных операций:

- Расширение (Expansion, E)

32-битный блок R расширяется до 48 бит с помощью функции расширения E , которая дублирует некоторые биты согласно фиксированной таблице: $E = [32, 1, 2, 3, 4, 5, 4, 5, 6, 7, 8, 9, 8, 9, 10, 11, 12, 13, 12, 13, 14, 15, 16, 17, 16, 17, 18, 19, 20, 21, 20, 21, 22, 23, 24, 25, 24, 25, 26, 27, 28, 29, 28, 29, 30, 31, 32, 1]$.

- Наложение ключа (Key Mixing)

Расширенный 48-битный блок складывается по модулю 2 (XOR) с 48-битным раундовым ключом K_i :

$$A = E(R) \oplus K_i$$

- Подстановка (Substitution, S-boxes)

Результат предыдущей операции A разделяется на восемь 6-битных блоков $A_1, A_2, \dots, A_8$. Каждый блок A_j обрабатывается соответствующей таблицей подстановки (S-box) S_j , преобразуя 6 бит входа в 4 бита выхода.

S-box представляет собой таблицу размером 4×16 , где первый и последний биты 6-битного входа определяют номер строки (0-3), а четыре средних бита — номер столбца (0-15). S-box вносят нелинейность в алгоритм и являются критически важными для его стойкости.

Например, S-box S_1 : $S_1 = [[14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7], [0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8], [4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0], [15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13]]$. Выход всех восьми S-box объединяется в 32-битный блок $B = B_1 \parallel B_2 \parallel \dots \parallel B_8$.

- Перестановка (Permutation, P)

Финальная перестановка P переставляет 32 бита результата S-box согласно таблице: $P = [16, 7, 20, 21, 29, 12, 28, 17, 1, 15, 23, 26, 5, 18, 31, 10, 2, 8, 24, 14, 32, 27, 3, 9, 19, 13, 30, 6, 22, 11, 4, 25]$.

Итоговый результат раундовой функции:

$$f(R, K_i) = P(S(E(R) \oplus K_i))$$

Генерация раундовых ключей

Из 64-битного основного ключа генерируются 16 раундовых ключей по 48 бит каждый. Процесс включает следующие шаги:

- Перестановка выбора ключа 1 (PC-1)

Из 64-битного ключа выбираются 56 значащих бит (8 бит четности отбрасываются) и переставляются согласно таблице PC-1, после чего делятся на две 28-битные половины C_0 и D_0 . PC-1 = [57, 49, 41, 33, 25, 17, 9, 1, 58, 50, 42, 34, 26, 18, 10, 2, 59, 51, 43, 35, 27, 19, 11, 3, 60, 52, 44, 36, 63, 55, 47, 39, 31, 23, 15, 7, 62, 54, 46, 38, 30, 22, 14, 6, 61, 53, 45, 37, 29, 21, 13, 5, 28, 20, 12, 4].

- Циклические сдвиги

Для каждого раунда i (от 1 до 16) обе половины C_{i-1} и D_{i-1} циклически сдвигаются влево на определенное число позиций согласно расписанию сдвигов: Сдвиги = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1].

Для раундов 1, 2, 9, 16 сдвиг производится на 1 бит, для остальных — на 2 бита.

$$C_i = LS_i(C_{i-1})$$

$$D_i = LS_i(D_{i-1})$$

где LS_i — операция циклического сдвига влево на число позиций, заданное для раунда i .

- Перестановка выбора ключа 2 (PC-2)

Объединенные 56-битные половины $C_i || D_i$ сжимаются до 48 бит и переставляются согласно таблице PC-2, формируя раундовый ключ K_i : PC-2 = [14, 17, 11, 24, 1, 5, 3, 28, 15, 6, 21, 10, 23, 19, 12, 4, 26, 8, 16, 7, 27, 20, 13, 2, 41, 52, 31, 37, 47, 55, 30, 40, 51, 45, 33, 48, 44, 49, 39, 56, 34, 53, 46, 42, 50, 36, 29, 32]

Процесс расшифрования

Благодаря структуре сети Фейстеля расшифрование выполняется тем же алгоритмом, что и шифрование, но с использованием раундовых ключей в обратном порядке: $K_{16}, K_{15}, \dots, K_1$. Это свойство называется инволютивностью преобразования Фейстеля.

Криптографическая стойкость

Теоретическая стойкость DES определяется длиной ключа в 56 бит, что дает пространство ключей размером $2^{56} \approx 7.2 \times 10^{16}$ возможных ключей. К концу 1990-х годов с развитием вычислительных мощностей стало возможным

проведение атак полного перебора на DES за разумное время. В 1998 году машина Deep Crack, созданная Electronic Frontier Foundation, смогла подобрать DES-ключ менее чем за 3 дня. Основные известные атаки на DES включают:

- Атака полного перебора — перебор всего пространства ключей
- Дифференциальный криптоанализ — анализ различий в парах открытых текстов
- Линейный криптоанализ — поиск линейных аппроксимаций нелинейных компонентов

Для повышения стойкости DES был разработан Triple DES (3DES), применяющий алгоритм DES трижды с двумя или тремя различными ключами, что увеличивает эффективную длину ключа до 112 или 168 бит.

Математические свойства

DES обладает рядом важных математических свойств:

1. Свойство дополнения — если $E_K(P) = C$, то $E_{\bar{K}}(\bar{P}) = \bar{C}$, где черта обозначает побитовое дополнение
2. Отсутствие слабых ключей в строгом смысле, хотя существуют полуслабые ключи и ключи с определенными симметриями
3. Лавинный эффект — изменение одного бита входа или ключа приводит к изменению в среднем половины битов выхода

TripleDES

Предпосылки создания и стандартизация

Triple Data Encryption Standard (3DES), также известный как TDEA (Triple Data Encryption Algorithm), был разработан в конце 1970-х — начале 1980-х годов как ответ на растущие опасения относительно недостаточной длины ключа оригинального DES. С развитием вычислительных технологий 56-битный ключ DES становился все более уязвимым к атакам полного перебора. Вместо разработки принципиально нового алгоритма, было предложено элегантное решение — каскадное применение проверенного DES с несколькими ключами.

Алгоритм 3DES был стандартизирован NIST в документе FIPS 46-3 (1999) и Special Publication 800-67, а также включен в международные стандарты ISO/IEC

18033-3. Несмотря на последующее принятие AES в 2001 году, 3DES продолжает широко использоваться в финансовых транзакциях, системах электронных платежей (EMV чипы банковских карт), защите данных спутниковой связи и legacy-системах. В 2017 году NIST начал постепенное выведение 3DES из употребления, установив окончательный срок прекращения использования на 2023 год для новых приложений.

Принцип работы алгоритма

3DES применяет алгоритм DES последовательно три раза к каждому 64-битному блоку данных, используя либо два, либо три различных ключа. Ключевая идея заключается в том, что многократное применение шифрования существенно увеличивает эффективную длину ключа и, следовательно, криптографическую стойкость.

Базовая операция 3DES описывается следующим образом:

$$C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$$

где:

- P — блок открытого текста (64 бита)
- C — блок шифртекста (64 бита)
- E_K — функция шифрования DES с ключом K
- D_K — функция расшифрования DES с ключом K
- K_1, K_2, K_3 — ключи DES

Схема "шифрование-расшифрование-шифрование" (EDE — Encrypt-Decrypt-Encrypt) была выбрана для обеспечения обратной совместимости с обычным DES: при использовании одинаковых ключей $K_1 = K_2 = K_3$, операция расшифрования в середине отменяет первое шифрование, и алгоритм вырождается в простой DES.

Процесс расшифрования выполняется в обратном порядке:

$$P = D_{K_1}(E_{K_2}(D_{K_3}(C)))$$

Варианты использования ключей

Стандарт 3DES определяет три варианта использования ключей, отличающихся по уровню безопасности и совместимости:

- 3TDES, три независимых ключа

Используются три независимых 56-битных ключа: $K_1 \neq K_2 \neq K_3$. Это обеспечивает максимальную теоретическую стойкость:

- Общая длина ключевого материала: 168 бит (3×56)
- Эффективная криптографическая стойкость: около 112 бит (из-за meet-in-the-middle атак)
- Схема: $E_{K_1}(D_{K_2}(E_{K_3}(P)))$

Этот вариант предлагает наивысший уровень безопасности среди реализаций 3DES, но редко используется на практике из-за управления тремя отдельными ключами.

- 2TDES, два независимых ключа

Наиболее распространенный вариант, где используются два независимых ключа: $K_1 \neq K_2$, но $K_3 = K_1$:

- Общая длина ключевого материала: 112 бит (2×56)
- Эффективная криптографическая стойкость: около 80 бит (с учетом улучшенных meet-in-the-middle атак)
- Схема: $E_{K_1}(D_{K_2}(E_{K_1}(P)))$

Этот вариант обеспечивает разумный баланс между безопасностью и практичностью, широко применялся в банковских системах и платежных терминалах. Однако NIST в SP 800-131A (2015) признал его устаревшим для защиты данных, требующих долгосрочной конфиденциальности.

- Одинаковые ключи

Все три ключа совпадают: $K_1 = K_2 = K_3$:

- Эффективная длина ключа: 56 бит
- Схема: $E_K(D_K(E_K(P))) = E_K(P)$

Этот вариант эквивалентен обычному DES и используется исключительно для обеспечения обратной совместимости со старыми системами. Не рекомендуется для использования в новых приложениях из-за низкой криптографической стойкости.

Альтернативная схема ЕЕЕ

Помимо основной схемы EDE, существует альтернативная схема ЕЕЕ (Encrypt-Encrypt-Encrypt):

$$C = E_{K_3}(E_{K_2}(E_{K_1}(P)))$$

Эта схема применяет шифрование DES три раза подряд без промежуточного расшифрования. Хотя схема ЕЕЕ теоретически обладает схожей стойкостью с EDE, она не обеспечивает обратную совместимость с обычным DES и поэтому не была включена в стандарт.

Криптоанализ и безопасность

Основная угроза для 3DES — это обобщенная meet-in-the-middle атака, впервые описанная Меркле и Хеллманом для двойного DES, а затем адаптированная для тройного. Для варианта с двумя ключами (Keying Option 2) атака работает следующим образом:

1. Атакующий имеет пары известных открытых текстов и шифротекстов: (P, C)
2. Для всех возможных K_1 вычисляется $X = E_{K_1}(P)$ и сохраняется в таблице
3. Для всех возможных K_2 вычисляется $Y = D_{K_2}(D_{K_1}(C))$ и ищется совпадение с таблицей
4. При совпадении найдена кандидатная пара ключей

Сложность атаки составляет $O(2^{56} + 2^{56}) = O(2^{57})$ операций при $O(2^{56})$ памяти, что снижает эффективную стойкость 112-битного ключа до примерно 80 бит. Для варианта с тремя ключами (Keying Option 1) улучшенная атака требует $O(2^{112})$ операций, обеспечивая эффективную стойкость около 112 бит.

Другие атаки:

- Chosen-plaintext атаки: требуют нереалистично большого количества данных (2^{90} выбранных открытых текстов) и практически неосуществимы
- Related-key атаки: теоретически возможны, но требуют контроля над выбором ключей, что маловероятно в реальных системах

- Sweet32 атака (2016): birthday-атака на 64-битные блоки, показавшая, что шифрование более 2^{32} блоков (32 ГБ) одним ключом создает уязвимости в протоколах TLS

Производительность

3DES требует примерно втрое больше вычислительных ресурсов по сравнению с обычным DES:

- Количество раундов: 48 (16 раундов $\times$ 3 прохода)
- Производительность: типично в 3 раза медленнее DES
- Пропускная способность: на современном процессоре около 50-100 МБ/с (программная реализация)
- Сравнение с AES: AES-128 примерно в 3-4 раза быстрее 3DES при сопоставимой или лучшей стойкости

Именно низкая производительность стала одной из причин постепенного отказа от 3DES в пользу AES.

Режимы работы

Как и обычный DES, 3DES может использоваться в различных режимах работы блочных шифров:

- ECB (Electronic Codebook) — независимое шифрование блоков
- CBC (Cipher Block Chaining) — каждый блок XOR'ится с предыдущим шифртекстом
- PCBC (Propagating Cipher Block Chaining) — модификация CBC, где каждый блок XOR'ится с предыдущими открытым текстом и шифртекстом
- CFB (Cipher Feedback) — поточный режим на основе блочного шифра
- OFB (Output Feedback) — генерация гаммы для XOR с открытым текстом
- CTR (Counter) — шифрование счетчика для получения гаммы
- RandomDelta — режим с добавлением случайной дельты к каждому блоку перед шифрованием

DEAL

Происхождение и контекст разработки

DEAL (Data Encryption Algorithm with Larger blocks) — экспериментальный блочный шифр, предложенный Lars Knudsen в 1998 году как кандидат на конкурс Advanced Encryption Standard (AES), проводимый NIST. Алгоритм представляет собой попытку создания нового блочного шифра на основе хорошо изученного и проверенного временем алгоритма DES, используя его в качестве базового строительного блока для конструкции с увеличенным размером блока.

Основная идея DEAL заключалась в использовании DES как примитива внутри структуры Фейстеля для обработки блоков большего размера (128 или 256 бит), что теоретически должно было обеспечить лучшую безопасность по сравнению с 64-битными блоками оригинального DES. Однако алгоритм не был принят в качестве стандарта AES и не получил широкого практического применения из-за выявленных криптоаналитических слабостей и недостаточной производительности.

Основная структура алгоритма

DEAL использует классическую структуру сети Фейстеля и существует в трех вариантах:

- DEAL-128: размер блока 128 бит, длина ключа 128 бит, 6 раундов
- DEAL-192: размер блока 128 бит, длина ключа 192 бит, 8 раундов
- DEAL-256: размер блока 128 бит, длина ключа 256 бит, 8 раундов

В базовой версии DEAL-128 входной блок данных разделяется на два 64-битных подблока L_0 и R_0 . Каждый раунд применяет следующее преобразование:

$$\begin{aligned} L_i &= R_{i-1} \\ R_i &= L_{i-1} \oplus F(R_{i-1}, K_i) \end{aligned}$$

где F — раундовая функция, основанная на DES.

Раундовая функция

Ключевой особенностью DEAL является использование DES в качестве раундовой функции. Раундовая функция $F(R, K_i)$ определяется следующим образом:

$$F(R, K_i) = DES_{K_i}(R) \oplus R$$

где:

- R — 64-битный правый подблок данных
- K_i — 56-битный раундовый ключ
- $DES_{K_i}(R)$ — результат шифрования блока R алгоритмом DES с ключом K_i
- $\oplus$ — операция побитового XOR

Эта конструкция известна как Davies-Meyer схема и добавляет операцию XOR с входным значением для усиления диффузии. Использование DES внутри раундовой функции позволяет наследовать его криптографические свойства: нелинейность S-box, перестановки и проверенную временем стойкость.

Генерация раундовых ключей

Схема расширения ключа в DEAL относительно проста и зависит от варианта алгоритма:

Для DEAL-128: из 128-битного основного ключа K генерируются 6 раундовых ключей по 56 бит каждый:

$$K_i = K[64(i - 1) : 64(i - 1) + 56], i = 1, 2, \dots, 6$$

То есть основной ключ разбивается на части, из которых извлекаются 56-битные подключи для использования в DES. Некоторые биты ключа могут использоваться повторно или применяться циклические перестановки для получения всех 6 раундовых ключей.

Для DEAL-256: аналогично, 256-битный ключ разделяется для формирования 8 раундовых ключей. Схема расширения может включать дополнительные операции перестановки или хэширования для обеспечения независимости раундовых ключей.

Количество раундов

Количество раундов в DEAL было определено исходя из анализа безопасности:

- DEAL-128: 6 раундов для обеспечения достаточной стойкости против известных атак
- DEAL-192 и DEAL-256: 8 раундов для учета увеличенного размера блока

Выбор числа раундов представляет собой компромисс между производительностью и безопасностью. Каждый раунд требует выполнения полного цикла DES, что делает DEAL относительно медленным алгоритмом.

Процесс шифрования и расшифрования

Шифрование на примере DEAL-128:

1. Разделить 128-битный блок открытого текста на L_0 и R_0 (по 64 бита)
2. Для каждого раунда i от 1 до 6:
 - $L_i = R_{i-1}$
 - $R_i = L_{i-1} \oplus (DES_{K_i}(R_{i-1}) \oplus R_{i-1})$
3. Объединить L_6 и R_6 для получения 128-битного шифртекста

Расшифрование: благодаря структуре Фейстеля расшифрование выполняется тем же алгоритмом с использованием раундовых ключей в обратном порядке: $K_6, K_5, \dots, K_1$.

Криптографические атаки

- Related-key атаки

В 1998 году John Kelsey, Bruce Schneier и David Wagner продемонстрировали эффективные related-key атаки на DEAL. Эти атаки эксплуатируют относительную простоту схемы расширения ключа, позволяя атакующему, имеющему доступ к шифрованию с несколькими связанными ключами, восстановить основной ключ быстрее, чем при полном переборе. Для DEAL-128 related-key атака требует примерно 2^{72} операций вместо теоретических 2^{128} , что существенно снижает эффективную стойкость.

- Slide атаки

Структура DEAL с относительно простым расписанием ключей делает его потенциально уязвимым к slide-атакам, где атакующий ищет пары блоков, связанных через определенное число раундов.

- Interpolation атаки

Использование DES как черного ящика внутри раундовой функции ограничивает применимость некоторых современных криптоаналитических методов, но при этом открывает возможности для специфических атак на структуру Фейстеля с малым числом раундов.

Производительность

Производительность DEAL значительно уступает как оригинальному DES, так и другим кандидатам AES:

- Скорость шифрования: примерно в 6 раз медленнее DES для DEAL-128 (6 раундов $\times$ 1 вызов DES на раунд)
- Аппаратная реализация: требует многократного использования аппаратного блока DES
- Программная реализация: накладные расходы на многократные вызовы DES-функции
- Сравнение с AES: типичная реализация DEAL в 10-15 раз медленнее AES

RSA

История создания и значение

RSA — первый практически применимый алгоритм асимметричного (публичного) шифрования, разработанный в 1977 году Рональдом Ривестом (Ron Rivest), Ади Шамиром (Adi Shamir) и Леонардом Адлеманом (Leonard Adleman) в Массачусетском технологическом институте (MIT). Алгоритм назван по первым буквам фамилий его создателей и стал революционным достижением в криптографии, решив фундаментальную проблему распределения ключей, существовавшую в симметричных криптосистемах.

RSA основан на математической задаче факторизации больших целых чисел — процессе разложения составного числа на простые множители. Хотя умножение двух больших простых чисел выполняется быстро, обратная операция (факторизация их произведения) является вычислительно сложной задачей, на которой и базируется стойкость алгоритма. На протяжении более 45 лет RSA остается одним из наиболее широко используемых криптографических алгоритмов в мире.

Математические основы

RSA опирается на несколько ключевых теорем теории чисел:

- Функция Эйлера $\varphi(n)$ определяет количество целых чисел в интервале от 1 до n , взаимно простых с n . Для двух различных простых чисел p и q :

$$\varphi(n) = \varphi(pq) = (p - 1)(q - 1)$$

- Теорема Эйлера утверждает, что для любого целого a , взаимно простого с n :

$$a^{\varphi(n)} \equiv 1(\text{mod } n)$$

- Малая теорема Ферма (частный случай теоремы Эйлера для простого p):

$$a^{p-1} \equiv 1(\text{mod } p)$$

На этих теоремах основан механизм шифрования и расшифрования RSA.

RSA использует операции в кольце вычетов по модулю n . Ключевые операции:

- Модульное возведение в степень: $c = m^e \text{ mod } n$
- Расширенный алгоритм Евклида: для нахождения мультипликативного обратного
- Модульное обратное: $d \equiv e^{-1}(\text{mod } \varphi(n))$, такое что $ed \equiv 1(\text{mod } \varphi(n))$

Генерация ключей

Процесс генерации пары ключей RSA состоит из следующих шагов:

- Выбор простых чисел

Генерируются два больших различных простых числа p и q . В современных реализациях используются числа длиной 1024-4096 бит каждое.

- Вычисление модуля

Вычисляется модуль RSA: $n = p \times q$. Число n используется как модуль для операций шифрования и расшифрования. Его длина определяет размер ключа RSA

- Вычисление функции Эйлера

$$\varphi(n) = (p - 1)(q - 1)$$

Значение $\varphi(n)$ критически важно для генерации ключей и должно сохраняться в секрете.

- Выбор открытой экспоненты

Выбирается целое число e такое, что:

$$1 < e < \varphi(n)$$

$$\gcd(e, \varphi(n)) = 1$$

Наиболее часто используются значения $e = 3$, $e = 17$ или $e = 65537 = 2^{16} + 1$ (четвертое число Ферма, F_4). Значение 65537 стало де-факто стандартом, поскольку обеспечивает баланс между безопасностью и вычислительной эффективностью.

- Выбор закрытой экспоненты

Вычисляется d — мультипликативное обратное к e по модулю $\varphi(n)$:

$$d \equiv e^{-1}(\text{mod } \varphi(n))$$

То есть:

$$ed \equiv 1(\text{mod } \varphi(n))$$

Для вычисления d используется расширенный алгоритм Евклида.

Результат:

- Открытый ключ: пара (e, n)
- Закрытый ключ: пара (d, n) или тройка (d, p, q)

Числа p , q , d и $\varphi(n)$ должны храниться в секрете и уничтожаться после генерации ключа (кроме d , необходимого для расшифрования).

Процесс шифрования

Для шифрования сообщения M (где $0 \leq M < n$) с использованием открытого ключа (e, n) :

$$C = M^e \bmod n$$

где:

- M — числовое представление открытого текста
- C — шифртекст
- e — открытая экспонента
- n — модуль RSA

Алгоритм:

1. Представить сообщение как целое число M
2. Убедиться, что $M < n$
3. Вычислить $C = M^e \bmod n$ с использованием быстрого алгоритма модульного возведения в степень
4. Передать шифртекст C

Процесс расшифровки

Для расшифрования шифртекста C с использованием закрытого ключа (d, n) :

$$M = C^d \bmod n$$

Доказательство корректности:

$$C^d \equiv (M^e)^d \equiv M^{ed} \pmod{n}$$

Так как $ed \equiv 1 \pmod{\varphi(n)}$, существует целое k такое, что:

$$ed = 1 + k\varphi(n)$$

Тогда:

$$M^{ed} = M^{1+k\varphi(n)} = M \cdot (M^{\varphi(n)})^k$$

По теореме Эйлера, если $\gcd(M, n) = 1$, то $M^{\varphi(n)} \equiv 1 \pmod{n}$, следовательно:

$$M^{ed} \equiv M \cdot 1^k \equiv M \pmod{n}$$

Оптимизация с китайской теоремой об остатках

Расшифрование и подписание RSA могут быть ускорены примерно в 4 раза с использованием Китайской теоремы об остатках при наличии факторов p и q :

$$M_p = C^{d_p} \bmod p, \text{ где } d_p = d \bmod (p-1)$$

$$M_q = C^{d_q} \bmod q, \text{ где } d_q = d \bmod (q-1)$$

Затем применяется CRT для восстановления M :

$$M = (M_p \cdot q \cdot q_p^{-1} + M_q \cdot p \cdot p_q^{-1}) \bmod n$$

где q_p^{-1} — обратное к q по модулю p , а p_q^{-1} — обратное к p по модулю q .

Криптоаналитические атаки

- Факторизация модуля

Основная атака — факторизация $n = pq$. При успешной факторизации можно вычислить $\varphi(n)$ и затем закрытый ключ d . Современные алгоритмы факторизации:

- Решето числового поля (GNFS): наиболее эффективный для больших чисел
- Метод эллиптических кривых (ECM): эффективен для специальных форм простых чисел
- Квантовый алгоритм Шора: теоретическая угроза, требует масштабируемого квантового компьютера
- Атака на малые экспоненты

- Атака на малое e : при $e = 3$ и малом сообщении возможно извлечение кубического корня
- Атака Хастада: при отправке одного сообщения с $e = 3$ нескольким получателям
- Атака Винера: при малом $d < n^{0.292}$ возможно восстановление закрытого ключа

Современное применение

RSA используется в:

- SSL/TLS: обмен ключами и аутентификация серверов
- SSH: аутентификация пользователей
- PGP/GPG: шифрование и подпись электронной почты
- Цифровые сертификаты: X.509 инфраструктура PKI
- Криптовалюты: некоторые блокчейн-системы
- Гибридные системы: RSA для обмена симметричными ключами, AES для шифрования данных

Rijndael

История создания и конкурс AES

Rijndael (произносится как "Рейндал") — симметричный блочный шифр, разработанный бельгийскими криптографами Жоаном Дэменом (Joan Daemen) и Винсентом Рэйменом (Vincent Rijmen) и принятый в 2001 году Национальным институтом стандартов и технологий США (NIST) в качестве Advanced Encryption Standard (AES).

В 1997 году NIST объявил открытый международный конкурс на замену устаревшему стандарту DES. Из 15 кандидатов, представленных в 1998 году, в финал вышли пять алгоритмов: MARS (IBM), RC6 (RSA Labs), Rijndael (Daemen & Rijmen), Serpent (Anderson, Biham, Knudsen) и Twofish (Schneier и др.). После трех лет интенсивного анализа криптографическим сообществом в октябре 2000 года Rijndael был единогласно выбран победителем благодаря оптимальному сочетанию безопасности, производительности, гибкости и простоты реализации.

Алгоритм был официально утвержден как федеральный стандарт FIPS PUB 197 в ноябре 2001 года и с тех пор стал наиболее широко используемым симметричным шифром в мире, применяясь в протоколах SSL/TLS, VPN, шифровании дисков, беспроводных сетях (WPA2/WPA3) и бесчисленном множестве других приложений.

Основные параметры

Rijndael является семейством блочных шифров с переменными размерами блока и ключа. Стандарт AES определяет следующие параметры:

- Размер блока: 128 бит (фиксированный для AES)
- Размеры ключа: 128, 192 или 256 бит
- Количество раундов:
 - 10 раундов для 128-битного ключа (AES-128)
 - 12 раундов для 192-битного ключа (AES-192)
 - 14 раундов для 256-битного ключа (AES-256)

Оригинальный Rijndael поддерживает также размеры блока 160 и 224 бит, но эти варианты не включены в стандарт AES.

Математическая основа

Rijndael построен на элегантной математической структуре — арифметике в поле Галуа $GF(2^8)$. Все байтовые операции интерпретируются как операции над элементами конечного поля с 256 элементами, где элементы представлены как полиномы степени меньше 8 с коэффициентами из $GF(2)$. Неприводимый полином, используемый для определения поля $GF(2^8)$:

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

Эта математическая структура обеспечивает хорошие криптографические свойства преобразований и упрощает доказательство стойкости алгоритма.

Структура алгоритма

Rijndael использует структуру Substitution-Permutation Network (SPN) — сеть подстановок и перестановок, в отличие от сети Фейстеля, применяемой в DES. В

этой структуре все биты блока обрабатываются параллельно в каждом раунде, что обеспечивает лучшую диффузию. Данные представляются в виде двумерного массива байтов, называемого State (состояние), размером 4×4 байта (для 128-битного блока). Байты заполняются по столбцам: первые 4 байта входного блока образуют первый столбец, следующие 4 — второй столбец и т.д.

Раундовые преобразования

Каждый раунд Rijndael (кроме последнего) состоит из четырех последовательных преобразований:

- SubBytes (Подстановка байтов)

Нелинейная операция подстановки, применяемая независимо к каждому байту состояния. Каждый байт заменяется согласно фиксированной таблице подстановки (S-box).

S-box Rijndael строится в два этапа:

1. Мультипликативная инверсия в поле $GF(2^8)$: байт b заменяется на b^{-1} (0 отображается в 0)
2. Аффинное преобразование: применяется битовая матрица и сложение с константой

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i$$

где $c = 0x63$ — константа аффинного преобразования.

Этот S-box был специально разработан для обеспечения:

- Нелинейности (высокое расстояние от любых линейных и аффинных функций)
- Отсутствия неподвижных точек ($S(a) \neq a$)
- Отсутствия обратных неподвижных точек ($S(a) \neq \bar{a}$)
- Минимизации разностных характеристик
 - ShiftRows (Циклический сдвиг строк)

Операция перестановки, циклически сдвигающая строки массива состояния:

- 0-я строка (верхняя): не сдвигается

- 1-я строка: сдвиг влево на 1 байт
- 2-я строка: сдвиг влево на 2 байта
- 3-я строка: сдвиг влево на 3 байта

Эта операция обеспечивает диффузию между столбцами.

- MixColumns (Перемешивание столбцов)

Линейное преобразование, действующее независимо на каждый столбец состояния. Каждый столбец рассматривается как полином над $GF(2^8)$ и умножается по модулю $x^4 + 1$ на фиксированный полином:

$$c(x) = 03 \cdot x^3 + 01 \cdot x^2 + 01 \cdot x + 02$$

В матричной форме:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \times \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}$$

где умножение и сложение выполняются в $GF(2^8)$.

Эта матрица является Maximum Distance Separable (MDS) кодом, обеспечивающим оптимальное рассеивание: изменение одного входного байта влияет на все четыре выходных байта.

Важно: в последнем раунде операция MixColumns опускается.

- AddRoundKey (Добавление раундового ключа)

Каждый байт состояния складывается по модулю 2 (XOR) с байтом раундового ключа:

$$s'_{i,j} = s_{i,j} \oplus k_{i,j}^{(r)}$$

где $k^{(r)}$ — раундовый ключ для раунда r .

Расширение ключа

Из основного ключа (128, 192 или 256 бит) генерируются раундовые ключи для всех раундов плюс начальный `AddRoundKey`. Процесс различается в зависимости от размера ключа:

- Для AES-128:

Основной ключ составляет 4 слова по 32 бита (16 байт). Генерируется 11 раундовых ключей (44 слова).

Алгоритм расширения:

1. Первые 4 слова копируются из основного ключа: $W[0..3] = Key$

2. Для каждого следующего слова $W[i]$ где $i \geq 4$:

- Если $i \bmod 4 = 0$:
 - $temp = SubWord(RotWord(W[i - 1])) \oplus Rcon[i/4]$
 - $W[i] = W[i - 4] \oplus temp$
- Иначе:
 - $W[i] = W[i - 4] \oplus W[i - 1]$

где:

- `RotWord`: циклический сдвиг байтов слова влево на 1 позицию
- `SubWord`: применение S-box к каждому байту слова
- `Rcon[i]`: раундовая константа $[x^{i-1}, 00, 00, 00]$ в $GF(2^8)$

Раундовые константы: `Rcon = [01, 02, 04, 08, 10, 20, 40, 80, 1B, 36, ...]`

- Для AES-192 и AES-256:

Процесс аналогичен, но с модификациями для учета большей длины ключа. AES-256 дополнительно применяет `SubWord` каждые 8 слов для усиления диффузии.

Процесс шифрования

Полный процесс шифрования AES:

1. Начальное добавление ключа: $State = Plaintext \oplus RoundKey$
2. Раунды 1 до Nr-1:
 - SubBytes(State)
 - ShiftRows(State)
 - MixColumns(State)
 - AddRoundKey(State, RoundKey[round])
3. Финальный раунд Nr:
 - SubBytes(State)
 - ShiftRows(State)
 - AddRoundKey(State, RoundKey[Nr])
4. Результат: $Ciphertext = State$

Процесс расшифрования

Расшифрование выполняется обратными операциями в обратном порядке:

- InvSubBytes: использует обратную S-box
- InvShiftRows: циклические сдвиги вправо
- InvMixColumns: умножение на обратную матрицу MixColumns
- AddRoundKey: та же операция (XOR инволютивна)

Раундовые ключи применяются в обратном порядке.

Криптобезопасность и атаки

Rijndael был спроектирован с использованием **Wide Trail Strategy** — стратегии, обеспечивающей:

- Минимальное число активных S-box в любом дифференциальном или линейном следе
- Быстрое распространение изменений (лавинный эффект)

- Гарантированную стойкость против дифференциального и линейного криптоанализа

На сегодняшний день не существует практических атак на полнораундовый AES. Лучшие известные атаки:

- Related-key атаки:
- AES-192: атака на 10 из 12 раундов
- AES-256: атака на 13 из 14 раундов (Biryukov & Khovratovich, 2009)
- Атаки на уменьшенное число раундов:
- 7 раундов AES-128 могут быть атакованы с вычислительной сложностью 2^{99}
- 10 раундов AES-256 могут быть атакованы при related-key сценарии

Wiclique атака (2011): теоретическая атака на полный AES с немного лучшей сложностью, чем полный перебор, но практически неосуществимая:

- AES-128: $2^{126.1}$ операций
- AES-256: $2^{254.4}$ операций

RC4

История

RC4 (Rivest Cipher 4, также известный как ARC4 или ARCFOUR) — поточный (stream cipher) шифр, разработанный Рональдом Ривестом (Ronald Rivest) для компании RSA Security в 1987 году. Название "RC4" означает "Rivest Cipher 4" — четвертый в серии шифров, разработанных Ривестом (после RC2, который был вторым публично известным).

Алгоритм был коммерческой тайной RSA Security до 1994 года, когда его описание было анонимно опубликовано в списке рассылки Cypherpunks. Хотя RSA Security никогда официально не подтверждала, что опубликованный алгоритм является настоящим RC4, последующий анализ и тестирование показали, что он генерирует идентичные выходные данные. С тех пор RC4 стал одним из наиболее широко используемых поточных шифров в истории криптографии.

Основные характеристики

RC4 обладает следующими особенностями:

- Тип: поточный шифр (stream cipher)
- Размер ключа: переменный, от 40 до 2048 бит (обычно 128 или 256 бит)
- Внутреннее состояние: массив из 256 байт (S-box) плюс два индекса
- Размер выхода: 1 байт псевдослучайных данных за итерацию
- Операции: только сложение и обмен байтов (без умножения или битовых операций)
- Простота: один из самых простых криптографических алгоритмов для реализации

Простота и высокая скорость сделали RC4 популярным выбором для многих протоколов, включая SSL/TLS (до версии 1.2), WEP, WPA (TKIP) и Microsoft Point-to-Point Encryption.

Структура алгоритма

RC4 состоит из двух основных фаз:

1. KSA (Key Scheduling Algorithm) — инициализация внутреннего состояния на основе ключа
2. PRGA (Pseudo-Random Generation Algorithm) — генерация потока псевдослучайных байтов

KSA инициализирует внутреннее состояние алгоритма — массив S размером 256 байт и два индекса i и j . Массив S заполняется значениями от 0 до 255 по порядку (identity permutation). Далее он перемешивается с использованием ключа.

После инициализации PRGA генерирует поток псевдослучайных байтов, которые складываются по модулю 2 (XOR) с открытым текстом для получения шифртекста. Благодаря свойству операции XOR, шифрование и расшифрование выполняются одной и той же операцией.

Математические свойства

RC4 не основан на сложных математических структурах, в отличие от блочных шифров типа AES. Его безопасность базируется на:

- Размёре внутреннего состояния: $256! \approx 2^{1684}$ возможных перестановок массива S
- Непредсказуемости индекса j : значение j зависит от всех предыдущих операций
- Выходной функции: $K = S[(S[i] + S[j]) \bmod 256]$ создаёт сложную зависимость от состояния

Однако отсутствие строгого математического обоснования сделало RC4 уязвимым к различным криптоаналитическим атакам.

Атаки

За десятилетия исследований были обнаружены многочисленные слабости RC4:

- Связанные ключи (Related-key attacks):

Некоторые связанные ключи приводят к идентичным или сильно коррелированным перестановкам после KSA.

- Слабые ключи:

Определённые ключи создают предсказуемые паттерны в начальной перестановке S .

- Проблема первых байтов:

Первые несколько байтов выходного потока PRGA имеют статистические смещения и коррелируют с ключом. Наиболее известные:

- Второй байт имеет вероятность $1/128$ быть равным нулю (вместо $1/256$)
- Байты на позициях 3-255 имеют различные смещения

Рекомендация: отбрасывать первые 256 или даже 3072 байта выходного потока (эта модификация называется RC4-drop[n]).

LOKI97

История

LOKI97 — симметричный блочный шифр, разработанный австралийскими криптографами Lawrie Brown, Josef Pieprzyk и Jennifer Seberry из University of Wollongong в 1997 году как кандидат на конкурс Advanced Encryption Standard (AES), проводимый NIST. Алгоритм является преемником более ранних шифров семейства LOKI — LOKI89 и LOKI91, разработанных той же исследовательской группой.

Название "LOKI" происходит от имени скандинавского бога обмана и хитрости Локи, что отражает криптографическую природу алгоритма — скрывать истинное значение данных. Цифра "97" в названии указывает на год представления алгоритма на конкурс AES.

LOKI97 был одним из 15 алгоритмов-кандидатов первого раунда конкурса AES, но не прошел во второй раунд. Несмотря на это, алгоритм представляет интерес с академической точки зрения и демонстрирует альтернативный подход к проектированию блочных шифров.

Основные параметры

LOKI97 обладает следующими характеристиками:

- Размер блока: 128 бит (фиксированный)
- Размеры ключа: 128, 192 или 256 бит
- Количество раундов: 16 раундов (для всех размеров ключа)
- Структура: модифицированная сеть Фейстеля

Структура алгоритма

Входной 128-битный блок разделяется на две 64-битные половины. Структура Фейстеля работает следующим образом:

Для каждого раунда i (где $i = 1, 2, \dots, 16$):

Входные данные раунда: две 64-битные половины L_{i-1} и R_{i-1}

Раундовое преобразование:

$$R_i = L_{i-1} \oplus f(T, K_{i,b})$$

$$T = R_{i-1} + K_{i,a}$$

$$L_i = R_{i-1} + K_{i,c}$$

где:

- f — раундовая функция
- $K_{i,a}, K_{i,b}, K_{i,c}$ — три раундовых подключа
- $+$ — арифметическое сложение по модулю 2^{64}

В конце 16 раунда половинки не меняются местами.

Раундовая функция

Отвечает за конфузию (запутывание) и диффузию (перемешивание) битов. Состоит из двух слоев:

- Слой подстановки (S): использование S-блоков

Входное 64 битное значение разбивается на 8 байт. К каждому байту применяется операция замены с использованием таблиц замен (S-Box: S1 и S2). Распределение по байтам в следующем порядке: [S1, S2, S1, S2, S2, S1, S2, S1]. Перед попаданием в S-Box значение байта смешивается с соответствующим байтом ключа.

После подстановки получаем:

$$B = b_7 \parallel b_6 \parallel b_5 \parallel b_4 \parallel b_3 \parallel b_2 \parallel b_1 \parallel b_0$$

S-блоки — не случайный набор чисел. Они вычисляются математически в поле Галуа ($GF(2^8)$) по неприводимому полиному $P(x)$:

- S1

$$S1(x) = x^3(mod(P(x)))$$

- S2

$$S2(x) = x^{-1}(mod(P(x)))$$

где x^{-1} — мультипликативная инверсия.

- Слой перестановки (P): перемешивание битов

После замены байтов получается новое 64-битное число. К нему применяется битовая перестановка Р для обеспечения диффузии.

Перестановка Р разработана так, чтобы:

- Каждый выходной бит зависел от битов из разных S-box
- Обеспечивалась быстрая диффузия изменений по всему блоку
- Минимизировалась вероятность успешного дифференциального криптоанализа

Расписание ключей

Для каждого раунда i (от 1 до 16) подключ генерируется следующим образом:

$$SK_i = \begin{cases} K_L \lll (5i \bmod 64) & \text{если } i \text{ нечетное} \\ K_R \lll (5i \bmod 64) & \text{если } i \text{ четное} \end{cases}$$

где $\lll n$ обозначает циклический сдвиг влево на n битов.

Дешифровка

Тот же алгоритм, но:

$$L_{i-1} = R_i \oplus f(T, K_{i,b})$$

$$T = R_{i-1} + K_{i,a}$$

$$R_{i-1} = L_i - K_{i,c}$$

Раунды в обратном порядке (от 16 до 1), ключи соответственно тоже в обратном порядке применяются.

Криптостойкость

LOKI97 был первым опубликованным кандидатом в конкурсе Advanced Encryption Standard, был в достаточно краткие сроки анализиован и атакован. В работе «Weaknesses in LOKI97» (Rijmen & Knudsen, 1999) было выявлено, что алгоритм имеет ряд недостатков, которые делают его уязвимым для криптоанализа.

Согласно исследованиям, проведенным в рамках конкурса AES, изменение одного бита входных данных в одном из раундов с относительно высокой вероятностью повлечет за собой изменение 1 бита в выходных данных, что делает

дифференциальную атаку успешной за приемлемое время. В то же время несбалансированность F-функции делает успешным линейный криптоанализ при разумных сроках. В то же время автор в описании алгоритма оценивал стойкость LOKI97 на несколько порядков большей (предполагалось, что для взлома необходимо обладать как минимум 2^{70} шифротекстами). Этот анализ недостатков алгоритма не позволил шифру LOKI97 пройти в следующий тур конкурса AES.

Протокол Диффи-Хеллмана

История

До появления протокола Диффи-Хеллмана существовала фундаментальная проблема распределения ключей: как двум сторонам установить общий секретный ключ через незащищенный канал связи без предварительного обмена секретной информацией? Все симметричные криптосистемы требовали безопасной передачи ключа, что было серьезным практическим ограничением.

Диффи и Хеллман предложили элегантное решение, основанное на вычислительной сложности задачи дискретного логарифмирования. Их протокол позволял двум сторонам, никогда ранее не общавшимся и не имеющим общих секретов, установить общий секретный ключ через открытый канал связи.

Математические основы

Безопасность протокола Диффи-Хеллмана основана на задаче дискретного логарифмирования (Discrete Logarithm Problem, DLP):

Дано простое число p , первообразный корень g по модулю p и число $h = g^x \bmod p$. Найти показатель степени x (называемый дискретным логарифмом).

Асимметрия сложности:

- Прямая задача (возведение в степень):

вычислить $g^x \bmod p$ — легко, выполняется за полиномиальное время $O(\log x)$ с использованием быстрого алгоритма возведения в степень

- Обратная задача (логарифмирование):

найти x зная $g^x \bmod p$ — вычислительно сложно, лучшие известные алгоритмы имеют субэкспоненциальную сложность

Число g называется первообразным корнем по модулю простого p , если:

$$\{g^1 \bmod p, g^2 \bmod p, \dots, g^{p-1} \bmod p\} = \{1, 2, \dots, p-1\}$$

То есть степени g генерируют все ненулевые остатки по модулю p .

Свойства:

- Для любого простого p существуют $\phi(p-1)$ первообразных корней
- Степени первообразного корня образуют циклическую группу порядка $p-1$

Протокол работает в мультипликативной группе $\mathbb{Z}_p^*$ — множестве чисел от 1 до $p-1$ с операцией умножения по модулю p .

Для простого p :

- Группа $\mathbb{Z}_p^*$ является циклической
- Порядок группы: $|\mathbb{Z}_p^*| = p-1$
- Теорема Ферма: $a^{p-1} \equiv 1 \pmod{p}$ для любого a не кратного p

Классический Диффи-Хеллман

Открытые параметры выбираются публично и известны всем участникам:

- p — большое простое число (обычно 2048 или 3072 бит)
- g — первообразный корень по модулю p (генератор группы)

Эти параметры могут быть стандартизованы и использоваться многократно разными парами пользователей.

Про сам протокол обмена:

Участники: Алиса и Боб хотят установить общий секретный ключ.

Шаг 1: Генерация закрытых ключей

- Алиса выбирает случайное секретное число a , где $1 < a < p-1$
- Боб выбирает случайное секретное число b , где $1 < b < p-1$

Числа a и b хранятся в секрете и никогда не передаются.

Шаг 2: Вычисление открытых ключей

- Алиса вычисляет: $A = g^a \bmod p$
- Боб вычисляет: $B = g^b \bmod p$

Шаг 3: Обмен открытыми ключами

- Алиса отправляет A Бобу (через открытый канал)
- Боб отправляет B Алисе (через открытый канал)

Шаг 4: Вычисление общего секрета

- Алиса вычисляет: $K_A = B^a \bmod p = (g^b)^a \bmod p = g^{ab} \bmod p$
- Боб вычисляет: $K_B = A^b \bmod p = (g^a)^b \bmod p = g^{ab} \bmod p$

Результат:

$$K_A = K_B = K = g^{ab} \bmod p$$

Алиса и Боб получили общий секретный ключ K , который может использоваться для симметричного шифрования.

Математическое доказательство корректности

$$\begin{aligned} K_A &= B^a \bmod p = (g^b \bmod p)^a \bmod p = g^{ba} \bmod p \\ K_B &= A^b \bmod p = (g^a \bmod p)^b \bmod p = g^{ab} \bmod p \end{aligned}$$

По свойству коммутативности умножения: $ab = ba$, следовательно:

$$K_A = g^{ab} \bmod p = g^{ba} \bmod p = K_B$$

Выбор параметров

Размер простого числа p

Рекомендации (NIST SP 800-57, 2020):

- 1024 бит: устарел, не рекомендуется (эквивалент ~80 бит симметричной стойкости)
- 2048 бит: минимальный рекомендуемый для новых систем (~112 бит стойкости)

- 3072 бит: для долгосрочной защиты (~128 бит стойкости)
- 4096 бит и выше: максимальная защита (~150+ бит стойкости)

Форма простого числа:

Часто используются безопасные простые числа (safe primes): $p = 2q + 1$, где q также простое. Это упрощает выбор генератора и защищает от некоторых атак.

1.8.7.2. Выбор генератора g

Для безопасного простого $p = 2q + 1$:

- Часто используется $g = 2$ или малое простое число
- Проверяется, что $g^2 \equiv 1 \pmod{p}$ и $g^q \not\equiv 1 \pmod{p}$

Для общего простого p :

- Выбирается первообразный корень из множества кандидатов
- Проверяется порядок элемента

1.8.7.3. Стандартизованные группы

Для упрощения реализации используются стандартизованные параметры:

RFC 3526 (2003): определяет группы DH для размеров 1536, 2048, 3072, 4096, 6144, 8192 бит

Атаки

- Man-in-the-Middle атака (MITM)

Критическая уязвимость: базовый протокол Диффи-Хеллмана не обеспечивает аутентификацию.

Сценарий атаки:

1. Алиса вычисляет $A = g^a$ и отправляет Бобу
2. Злоумышленник Ева перехватывает A
3. Ева генерирует e , вычисляет $E = g^e$ и отправляет E Бобу (вместо A)
4. Боб вычисляет $B = g^b$ и отправляет Алисе

5. Ева перехватывает B

6. Ева отправляет E Алисе (вместо B)

Результат:

- Алиса и Ева установили ключ $K_{AE} = g^{ae}$
- Боб и Ева установили ключ $K_{BE} = g^{be}$
- Ева может расшифровывать и читать/модифицировать все сообщения

Защита:

Требуется аутентификация открытых ключей через:

- Цифровые подписи (RSA, ECDSA)
- Заранее распределенные сертификаты
- Аутентифицированный протокол (Authenticated DH)
- Атаки на слабые параметры

Атака на малые подгруппы (Small Subgroup Attack):

Если порядок подгруппы, генерируемой g , мал, атакующий может перебрать все возможные значения общего секрета.

Защита: использование безопасных простых и проверка порядка элементов.

- Атака Pohlig-Hellman:

Если $p - 1$ имеет только малые простые делители, задачу DLP можно решить эффективно через китайскую теорему об остатках.

Защита: использование безопасных простых $p = 2q + 1$.

Практическая часть

DES

Реализация DES представляет собой несколько модулей: библиотека с самим алгоритмом, директория с тестами, демо-версия в виде консольного приложения с возможностью выбора операции (шифрования, дешифрования), режима набивки и режима шифровки, приложение для полноценного запуска. Общая архитектура выглядит следующим образом:

Листинг 1. Общая архитектура

```
CryptoLab_DES/  
|  
├─ CryptoLib/  
|   ├─ Core/  
|   |   └─ BitPermutation.cs  
|   ├─ Interfaces/  
|   |   ├─ IKeyScheduler.cs  
|   |   ├─ IFeistelFunction.cs  
|   |   └─ ISymmetricCipher.cs  
|   └─ Algorithms/  
|       ├─ FeistelNetwork.cs  
|       ├─ FeistelNetworkDeal.cs  
|       └─ DES/  
|           ├─ DESTables.cs  
|           ├─ DESKeyScheduler.cs  
|           ├─ DESFeistelFunction.cs  
|           └─ DESAlgorithm.cs  
|       └─ DEAL/  
|           └─ DEALAlgorithm.cs
```

```

|   |   |   └─ DEALKeyScheduler.cs
|   |   |   └─ DESAdapter.cs
|   |   |   └─ DEALFeistelFunction.cs
|   |   |   └─ TripleDES/
|   |   |       └─ TripleDESAlgorithm.cs
|   |   └─ Modes/
|   |       └─ CipherContext.cs
|   |       └─ CipherMode.cs
|   |       └─ PaddingMode.cs
|   └─ CryptoLib.csproj
|
|
└─ CryptoDemo/
    └─ Program.cs
    └─ CryptoDemo.csproj
|
|
└─ CryptoTests/
    └─ BitPermutationTests.cs
    └─ DESTests.cs
    └─ DEALTests.cs
    └─ CryptoTests.csproj
|
|
└─ CryptoLab_DES.sln
|
└─ README.md

```

Реализация DES представляет собой класс, который собирает и конфигурирует все нужные параметры, после чего делегирует выполнение конкретных действий (установка раундовых ключей, выполнение 1 раунда функции Фейстеля и т.д.) конкретным классам реализаций.

Листинг 2. Класс DESAlgorithm.cs

```
using CryptoLib.DES.Interfaces;
```

```

namespace CryptoLib.DES.Algorithms.DES
{
    public class DESAlgorithm : ISymmetricCipher
    {
        private readonly FeistelNetwork _feistelNetwork;
        private bool _keysSet = false;
        public DESAlgorithm()
        {
            var keyScheduler = new DESKeyScheduler();
            var feistelFunction = new DESFeistelFunction();
            _feistelNetwork = new FeistelNetwork(keyScheduler,
feistelFunction, 16);
        }
        public int BlockSize => 8;
        public int KeySize => 8;
        public void SetRoundKeys(byte[] key)
        {
            if (key.Length != KeySize)
                throw new ArgumentException($"DES key must be
{KeySize} bytes");
            _feistelNetwork.SetRoundKeys(key);
            _keysSet = true;
        }
        public byte[] EncryptBlock(byte[] block)
        {
            if (!_keysSet)
                throw new InvalidOperationException("Round keys not
set. Call SetRoundKeys first.");
            if (block.Length != BlockSize)

```

```

        throw new ArgumentException($"Block must be
{BlockSize} bytes");
        return _feistelNetwork.EncryptBlock(block);
    }
    public byte[] DecryptBlock(byte[] block)
    {
        if (!_keysSet)
            throw new InvalidOperationException("Round keys not
set. Call SetRoundKeys first.");
        if (block.Length != BlockSize)
            throw new ArgumentException($"Block must be
{BlockSize} bytes");
        return _feistelNetwork.DecryptBlock(block);
    }
}
}

```

В конструкторе здесь инициализируется 1 экземпляр класса путем передачи конкретных реализаций всех частей, нужных для работы шифра. Затем во всех методах после проверок происходит делегирование функциональности методам из других классов.

Реализация сети Фейстеля представляет собой такой же класс:

Листинг 3. Класс FeistelNetwork.cs

```

using CryptoLib.DES.Interfaces;
namespace CryptoLib.DES.Algorithms
{
    public class FeistelNetwork : ISymmetricCipher
    {
        private readonly IKeyScheduler _keyScheduler;
        private readonly IFeistelFunction _feistelFunction;
        private readonly int _rounds;
    }
}

```

```

private byte[][] _roundKeys = default!;

public FeistelNetwork(
    IKeyScheduler keyScheduler,
    IFeistelFunction feistelFunction,
    int rounds)
{
    _keyScheduler = keyScheduler;
    _feistelFunction = feistelFunction;
    _rounds = rounds;
}

public int BlockSize => 8;
public int KeySize => 8;
public void SetRoundKeys(byte[] key)
{
    _roundKeys = _keyScheduler.ExpandKey(key);
}

public byte[] EncryptBlock(byte[] block)
{
    if (_roundKeys == null) throw new
InvalidOperationException("Round keys not set.");

    int halfSize = BlockSize / 2;
    byte[] left = new byte[halfSize];
    byte[] right = new byte[halfSize];
    Array.Copy(block, 0, left, 0, halfSize);
    Array.Copy(block, halfSize, right, 0, halfSize);
    for (int round = 0; round < _rounds; round++)
    {
        byte[] tempLeft = (byte[])left.Clone();

```

```

        left = right;

        byte[] feistelResult =
_feistelFunction.Execute(right, _roundKeys[round]);

        right = XorBlocks(tempLeft, feistelResult);
    }

    byte[] result = new byte[BlockSize];
    Array.Copy(left, 0, result, 0, halfSize);
    Array.Copy(right, 0, result, halfSize, halfSize);
    return result;
}

public byte[] DecryptBlock(byte[] block)
{
    if (_roundKeys == null) throw new
InvalidOperationException("Round keys not set.");

    int halfSize = BlockSize / 2;
    byte[] left = new byte[halfSize];
    byte[] right = new byte[halfSize];
    Array.Copy(block, 0, left, 0, halfSize);
    Array.Copy(block, halfSize, right, 0, halfSize);
    for (int round = _rounds - 1; round >= 0; round--)
    {
        byte[] tempRight = (byte[])right.Clone();
        right = left;
        byte[] feistelResult = _feistelFunction.Execute(left,
_roundKeys[round]);

        left = XorBlocks(tempRight, feistelResult);
    }

    byte[] result = new byte[BlockSize];
    Array.Copy(left, 0, result, 0, halfSize);

```

```

        Array.Copy(right, 0, result, halfSize, halfSize);
        return result;
    }
    private byte[] XorBlocks(byte[] block1, byte[] block2)
    {
        byte[] result = new byte[block1.Length];
        for (int i = 0; i < block1.Length; i++)
        {
            result[i] = (byte)(block1[i] ^ block2[i]);
        }
        return result;
    }
}

```

Здесь картина аналогичная: в конструкторе мы инициализируем 1 экземпляр сети Фейстеля, а затем в самих методах есть использование реализацией из других классов, например, классу DESKeyScheduler, который реализует планировщик ключей.

Все взаимодействие с DES осуществляется через класс CipherContext, где в том числе и реализованы все механизмы набивки и шифрования. Его часть с параметрами, конструктором и функцией по выбору алгоритма выглядит так:

Листинг 4. Список параметров, конструктор и выбор режима шифровки CipherContext.cs

```

public class CipherContext
{
    private readonly ISymmetricCipher _algorithm; // DES or DEAL
    private readonly CipherMode _mode; // выбранный режим
    private readonly object _algorithmLock = new object;
    private readonly PaddingMode _padding;
    private readonly byte[] _initializationVector;
    private readonly int _blockSize;

```

```

        private (byte[] Plaintext, byte[] Ciphertext)
        _pcbcFeedbackRegisters;

        private byte[] _feedbackRegister = default!;

        public CipherContext(
            byte[] key,
            CipherMode mode,
            PaddingMode padding,
            byte[]? initializationVector = null,
            params KeyValuePair<string, object>[] additionalParameters)
        {
            _mode = mode;
            _padding = padding;
            _initializationVector = initializationVector;
            _algorithm = CreateAlgorithm(key, additionalParameters);
            _algorithm.SetRoundKeys(key);
            _blockSize = _algorithm.BlockSize;
            ValidateParameters();
        }

        private void InitializeState()
        {
            if (_mode != CipherMode.ECB && _initializationVector != null)
            {
                _feedbackRegister = (byte[])_initializationVector.Clone();
                _pcbcFeedbackRegisters =
                (
                    (byte[])_initializationVector.Clone(),
                    (byte[])_initializationVector.Clone()
                );
            }
        }
    }

```



```

        private ISymmetricCipher CreateAlgorithm(byte[] key,
        KeyValuePair<string, object>[] additionalParameters)
        {
            if (additionalParameters != null)
            {
                var algorithmParam = additionalParameters.FirstOrDefault(p
=> p.Key == "Algorithm");
                if (!algorithmParam.Equals(default(KeyValuePair<string,
object>)))
                {
                    var algorithmValue = algorithmParam.Value.ToString();

                    if (algorithmValue.Contains("TripleDES") ||
algorithmValue.Contains("3DES"))
                    {
                        return new TripleDESAlgorithm();
                    }

                    if (algorithmValue.Contains("DEAL"))
                    {
                        return key.Length switch
                        {
                            16 => new
DEALAlgorithm(DEALKeyScheduler.KeyType.KEY_SIZE_128),
                            24 => new
DEALAlgorithm(DEALKeyScheduler.KeyType.KEY_SIZE_192),
                            32 => new
DEALAlgorithm(DEALKeyScheduler.KeyType.KEY_SIZE_256),
                            _ => throw new ArgumentException($"Unsupported
DEAL key size: {key.Length} bytes")
                        };
                    }
                }
            }
        }

```

```

        else if (algorithmValue.Contains("DES"))
        {
            return new DESAlgorithm();
        }
        return algorithmValue switch
        {
            "DEAL" => new DEALAlgorithm(),
            "DES" => new DESAlgorithm(),
            "TripleDES" => new TripleDESAlgorithm(),
            "3DES" => new TripleDESAlgorithm(),
            _ => throw new ArgumentException($"Unsupported
algorithm: {algorithmValue}")
        };
    }
}

return key.Length switch
{
    8 => new DESAlgorithm(),
    16 => new
DEALAlgorithm(DEALKeyScheduler.KeyType.KEY_SIZE_128),
    24 => new
DEALAlgorithm(DEALKeyScheduler.KeyType.KEY_SIZE_192),
    32 => new
DEALAlgorithm(DEALKeyScheduler.KeyType.KEY_SIZE_256),
    _ => throw new ArgumentException($"Unsupported key size:
{key.Length} bytes")
};
}

```

Таким образом, реализация DES представляет собой гибкое приложение, устойчивое к ошибкам, с возможностью изменения реализаций или добавления функциональности. А работоспособность представлена в разных формах: модульные и интеграционные тесты для всего приложения с выводом скорости

шифровки разных режимов и типов файлов, демо-версия в терминале и приложение на Avalonia.

TripleDES

Реализация TripleDES представляет собой лишь 1 файл, который обращается к описанному выше алгоритму DES и де-факто вызывает эти экземпляры и все нужные методы внутри себя. Архитектурно TripleDES представлен лишь еще 1 директорией с файлом TripleDESAlgorithm.cs внутри директории Algorithms в DES (см. листинг 1 с архитектурой). Сама реализация представляет собой лишь еще один класс:

Листинг 5. Класс TripleDESAlgorithms

```
using System;
using CryptoLib.DES.Interfaces;
using CryptoLib.DES.Algorithms.DES;
namespace CryptoLib.DES.Algorithms.TripleDES
{
    public class TripleDESAlgorithm : ISymmetricCipher
    {
        private readonly DESAlgorithm _des1;
        private readonly DESAlgorithm _des2;
        private readonly DESAlgorithm _des3;
        private bool _keysSet = false;
        public TripleDESAlgorithm()
        {
            _des1 = new DESAlgorithm();
            _des2 = new DESAlgorithm();
            _des3 = new DESAlgorithm();
        }
        public int BlockSize => 8;
        public int KeySize => 24;
        public void SetRoundKeys(byte[] key)
```

```

{
    if (key == null)
        throw new ArgumentNullException(nameof(key));
    if (key.Length != KeySize)
        throw new ArgumentException($"TripleDES key must be
{KeySize} bytes (192 bits).");
    byte[] key1 = new byte[8];
    byte[] key2 = new byte[8];
    byte[] key3 = new byte[8];
    Array.Copy(key, 0, key1, 0, 8);
    Array.Copy(key, 8, key2, 0, 8);
    Array.Copy(key, 16, key3, 0, 8);
    _des1.SetRoundKeys(key1);
    _des2.SetRoundKeys(key2);
    _des3.SetRoundKeys(key3);
    _keysSet = true;
}

public byte[] EncryptBlock(byte[] block)
{
    if (!_keysSet)
        throw new InvalidOperationException("Round keys not set.
Call SetRoundKeys first.");
    if (block.Length != BlockSize)
        throw new ArgumentException($"Block must be {BlockSize}
bytes");
    byte[] stage1 = _des1.EncryptBlock(block);
    byte[] stage2 = _des2.DecryptBlock(stage1);
    byte[] result = _des3.EncryptBlock(stage2);
    return result;
}

public byte[] DecryptBlock(byte[] block)

```

```

        {
            if (!_keysSet)
                throw new InvalidOperationException("Round keys not set.
Call SetRoundKeys first.");
            if (block.Length != BlockSize)
                throw new ArgumentException($"Block must be {BlockSize}
bytes");
            byte[] stage1 = _des3.DecryptBlock(block);
            byte[] stage2 = _des2.EncryptBlock(stage1);
            byte[] result = _des1.DecryptBlock(stage2);
            return result;
        }
    }
}

```

Обращение к алгоритму осуществляется через все CipherContext (см. листинг 4). Для тестирования функциональности также написан блок текстов по образцу тестов для DES, а для демонстрации имеется выделенное окно в общем приложении для всех лабораторных работ в корне курсового проекта. Таким образом видно, что реализация TripleDES представляет собой фактически создание 3 экземпляров DES и их вызов в режиме EDE с одной из 3 форм ключей.

DEAL

Так как «под капотом» данный алгоритм также использует DES, а также такие же режимы шифровки и набивки, то и размещен он рядом с последним. Вся реализация уместается в файлы внутри директории DEAL, а также в отдельную реализацию функции Фейстеля (см. листинг 1).

Сам класс DEALAlgorithm совпадает с DES с тем лишь исключением, что тут доступен выбор длины ключа:

Листинг 6. Конструктор DEALAlgorithm

```

public DEALAlgorithm(DEALKeyScheduler.KeyType keyType =
DEALKeyScheduler.KeyType.KEY_SIZE_128)
{
    _keyType = keyType;
}

```

```

var keyScheduler = new DEALKeyScheduler(keyType);
var feistelFunction = new DEALFeistelFunction();
int rounds = keyType switch
{
    DEALKeyScheduler.KeyType.KEY_SIZE_128 => 6,
    DEALKeyScheduler.KeyType.KEY_SIZE_192 => 8,
    DEALKeyScheduler.KeyType.KEY_SIZE_256 => 8,
    _ => throw new ArgumentException("Invalid key type")
};

_feistelNetwork = new FeistelNetworkDEAL(keyScheduler,
feistelFunction, rounds);
}

public int BlockSize => 16; // 128 бит для DEAL
public int KeySize => _keyType switch
{
    DEALKeyScheduler.KeyType.KEY_SIZE_128 => 16,
    DEALKeyScheduler.KeyType.KEY_SIZE_192 => 24,
    DEALKeyScheduler.KeyType.KEY_SIZE_256 => 32,
    _ => throw new InvalidOperationException("Unknown key type")
};

```

Основная функция, выполняющая преобразование раундовой функции над входным блоком, в данном случае просто вызывает готовый экземпляр класса DES:

Листинг 7. Функция Execute

```

public byte[] Execute(byte[] inputBlock, byte[] roundKey)
{
    DESAlgorithm des = _threadLocalDes.Value!;
    des.SetRoundKeys(roundKey);
    return des.EncryptBlock(inputBlock);
}

```

Непохожей на имеющуюся реализацию в DES является функция для генерации ключей. Она также обращается к экземпляру класса DES, но с обработкой имеющейся логики, а также, кроме того, с учетом длины ключа:

Листинг 8. Функция ExpandKey

```
public byte[][] ExpandKey(byte[] inputKey)
{
    ValidateKeySize(inputKey);
    byte[][] roundKeys = new byte[_rounds][];
    var des = new DESAlgorithm();
    byte[] roundCounterBlock = new byte[8];
    switch (_keyType)
    {
        case KeyType.KEY_SIZE_128:
        {
            byte[] k1 = GetSubKey(inputKey, 0, 8);
            byte[] k2 = GetSubKey(inputKey, 8, 8);
            XorWithConstant(k1, C1);
            XorWithConstant(k2, C2);
            for (int i = 0; i < _rounds; i++)
            {
                roundCounterBlock[7] = (byte)(i + 1);
                if (i % 2 == 0)
                {
                    des.SetRoundKeys(k1);
                }
                else
                {
                    des.SetRoundKeys(k2);
                }
                roundKeys[i] = des.EncryptBlock(roundCounterBlock);
            }
        }
    }
}
```

```

    }
    break;
}
case KeyType.KEY_SIZE_192:
{
    byte[] k1 = GetSubKey(inputKey, 0, 8);
    byte[] k2 = GetSubKey(inputKey, 8, 8);
    byte[] k3 = GetSubKey(inputKey, 16, 8);
    XorWithConstant(k1, C1);
    XorWithConstant(k2, C2);
    XorWithConstant(k3, C3);
    for (int i = 0; i < _rounds; i++)
    {
        roundCounterBlock[7] = (byte)(i + 1);

        des.SetRoundKeys((i % 3) switch
        {
            0 => k1,
            1 => k2,
            _ => k3
        });
        roundKeys[i] = des.EncryptBlock(roundCounterBlock);
    }
    break;
}
case KeyType.KEY_SIZE_256:
{
    for (int i = 0; i < _rounds; i++)
    {

```



```

byte[] subKey = GetSubKey(inputKey, (i % 4) * 8,
8);

roundKeys[i] = subKey;
}
break;
}
}
return roundKeys;
}

```

Таким образом, реализация DEAL во многом опирается на алгоритм DES, а потому, как и в случае с TripleDES, нет особого смысла выносить данный криптографический алгоритм в отдельные директории и создавать для него отдельную архитектуру. Вызов алгоритма все также осуществляется через CipherContext (см. листинг 4). Точно также DEAL включен в итоговое приложение, имеет демо-версию, набор тестов. А сама реализация, как и все остальные, описана с документацией.

RSA

Реализация данного алгоритма уже выполнена в качестве отдельной работы со своим набором папок (аналогичным по формату, но отличным от DES/TripleDES/DEAL по содержанию) и архитектурой:

Листинг 9. Архитектура RSA

```

Cryptography_RSA/
|
├─ Cryptography_RSA.sln
|
├─ CryptoLib/                                     # Проект: Библиотека классов
|  |
|  └─ Interfaces/
|     └─ ICryptoMathService.cs
|     └─ IPrimalityTest.cs
|     └─ IRsaService.cs

```

```

|   |   └─ WienerAttackService.cs
|   |
|   └─ Core/
|   |   └─ CryptoMathService.cs
|   |
|   └─ Primality/
|   |   └─ PrimalityTestBase.cs
|   |   └─ Implementations/
|   |       └─ FermatTest.cs
|   |       └─ SoloveyStrassenTest.cs
|   |       └─ MillerRabinTest.cs
|   |
|   └─ RSA/
|   |   └─ RsaService.cs
|   |   └─ RsaKeyGenerator.cs
|   |   └─ Models/
|   |       └─ RsaPublicKey.cs
|   |       └─ RsaPrivateKey.cs
|   |       └─ RsaKeyPair.cs
|   |
|   └─ Attacks/
|   |   └─ WienerAttackService.cs
|   |   └─ Models/
|   |       └─ ContinuedFraction.cs
|   |       └─ WienerAttackResult.cs
|   |
|   └─ Enums/
|   |   └─ PrimalityTestType.cs
|   |
|   └─ CryptoLib.csproj

```

```
|
|
├─ CryptoDemo/
|  |
|  ├─ Program.cs
|  ├─ Demos/
|  |   ├─ Task1_MathServiceDemo.cs
|  |   ├─ Task2_PrimalityTestsDemo.cs
|  |   ├─ Task3_RsaDemo.cs
|  |   └─ Task4_WienersAttackDemo.cs
|  |
|  └─ CryptoDemo.csproj
|
|
├─ CryptoTests/
|  |
|  ├─ Core/
|  |   └─ CryptoMathServiceTests.cs
|  |
|  ├─ Primality/
|  |   └─ PrimalityTests.cs
|  |
|  ├─ RSA/
|  |   ├─ RsaKeyGeneratorTests.cs
|  |   └─ RsaServiceTests.cs
|  |
|  ├─ Attacks/
|  |   └─ WienersAttackServiceTests.cs
|  |
|  └─ CryptoTests.csproj
|
└─ README.md
```

В основе всего алгоритма лежат математические операции, которые реализованы с применением BigInt-ов, что позволяет потенциально подавать на вход полиномы бесконечной размерности.

Листинг 10. Математические операции RSA

```
using CryptoLib.RSA.Interfaces;
using CryptoLib.RSA.Primality.Implementations;
using System.Numerics;
namespace CryptoLib.RSA.Core
{
    public class CryptoMathService : ICryptoMathService
    {
        public BigInteger Gcd(BigInteger a, BigInteger b)
        {
            a = BigInteger.Abs(a);
            b = BigInteger.Abs(b);
            while (b != 0)
            {
                var temp = b;
                b = a % b;
                a = temp;
            }
            return a;
        }

        public (BigInteger Gcd, BigInteger X, BigInteger Y)
        ExtendedGcd(BigInteger a, BigInteger b)
        {
            BigInteger old_x = 1, x = 0;
            BigInteger old_y = 0, y = 1;
            while (b != 0)
            {
                var quotient = a / b;
```

```

        var temp_b = b;
        b = a % b;
        a = temp_b;
        var temp_x = x;
        x = old_x - quotient * x;
        old_x = temp_x;
        var temp_y = y;
        y = old_y - quotient * y;
        old_y = temp_y;
    }
    if (a < 0)
    {
        a = -a;
        old_x = -old_x;
        old_y = -old_y;
    }
    return (a, old_x, old_y);
}

public BigInteger ModPow(BigInteger value, BigInteger exponent,
    BigInteger modulus)
{
    if (modulus <= 0)
        throw new ArgumentException("Модуль должен быть
положительным числом.", nameof(modulus));
    if (exponent < 0)
        throw new ArgumentException("Показатель степени не может
быть отрицательным.", nameof(exponent));
    if (modulus == 1)
        return 0;
    BigInteger result = 1;
    value %= modulus;

```

```

while (exponent > 0)
{
    if ((exponent & 1) == 1)
    {
        result = result * value % modulus;
    }
    exponent >>= 1;
    value = value * value % modulus;
}
return result;
}

public int LegendreSymbol(BigInteger a, BigInteger p)
{
    if (p < 3 || p % 2 == 0)
        throw new ArgumentException("Модуль p должен быть нечетным простым числом больше 2.", nameof(p));

    IPrimalityTest primalityTest = new MillerRabinTest(this);
    if (!primalityTest.IsPrime(p, 0.99999))
    {
        throw new ArgumentException("Модуль p должен быть простым числом.", nameof(p));
    }

    var exponent = (p - 1) / 2;
    var result = ModPow(a, exponent, p);
    if (a % p == 0)
    {
        return 0;
    }
    if (result == 1)
    {
        return 1;
    }

```

```

    }
    return -1;
}

public int JacobiSymbol(BigInteger a, BigInteger n)
{
    if (n <= 0 || n % 2 == 0)
        throw new ArgumentException("n должно быть положительным
нечетным числом.", nameof(n));
    if (n == 1) return 1;
    a %= n;
    if (a < 0) a += n;
    if (a == 0) return 0;
    int j = 1;
    while ((a & 1) == 0)
    {
        a >>= 1;
        var n_mod_8 = n % 8;
        if (n_mod_8 == 3 || n_mod_8 == 5)
        {
            j = -j;
        }
    }
    if ((a % 4 == 3) && (n % 4 == 3))
    {
        j = -j;
    }
    return j * JacobiSymbol(n % a, a);
}
}
}

```

Сама реализация RSA представлена классом RsaService, который как раз и делегирует выполнение всех операций представленному в листинге выше классу с математическими операциями, а также иным классам.

Листинг 11. Класс RsaService.cs

```
using System.Numerics;
using CryptoLib.RSA.Core;
using CryptoLib.RSA.Enums;
using CryptoLib.RSA.Interfaces;
using CryptoLib.RSA.RSA.Models;
namespace CryptoLib.RSA.RSA
{
    public class RsaService : IRsaService
    {
        private readonly RsaKeyGenerator _keyGenerator;
        private readonly ICryptoMathService _mathService;

        public RsaService(PrimalityTestType testType, double probability,
int bitLength)
        {
            _mathService = new CryptoMathService();
            _keyGenerator = new RsaKeyGenerator(testType, probability,
bitLength, _mathService);
        }

        public RsaKeyPair GenerateKeyPair()
        {
            return _keyGenerator.GenerateKeyPair(65537);
        }

        public BigInteger Encrypt(BigInteger message, RsaPublicKey key)
        {
            return _mathService.ModPow(message, key.E, key.N);
        }

        public BigInteger Decrypt(BigInteger ciphertext, RsaPrivateKey key)
```



```

        {
            return _mathService.ModPow(ciphertext, key.D, key.N);
        }
    }
}

```

В основе алгоритма Rsa, помимо математических операций, лежат также 3 теста простоты. В реализации они представлены в отдельной директории в качестве разных файлов и классов соответственно, каждый из которых может вызываться в зависимости от передаваемых в конструктор RsaService (см. листинг выше) параметров.

Листинг 12. Класс MillerRabinTest.cs

```

using System.Numerics;
using CryptoLib.RSA.Interfaces;
namespace CryptoLib.RSA.Primalty.Implementations
{
    public class MillerRabinTest : PrimalityTestBase
    {
        public MillerRabinTest(ICryptoMathService mathService) :
base(mathService)
        {
            protected override bool PerformSingleIteration(BigInteger number)
            {
                BigInteger d = number - 1;
                int s = 0;
                while ((d & 1) == 0) // // d % 2 == 0
                {
                    d >>= 1;
                    s++;
                }
                var witness = GenerateRandomBigInteger(2, number - 2);
            }
        }
    }
}

```

```

        var x = _mathService.ModPow(witness, d, number);
        if (x == 1 || x == number - 1)
        {
            return true;
        }
        for (int r = 1; r < s; r++)
        {
            x = _mathService.ModPow(x, 2, number);
            if (x == number - 1) return true;
        }
        return false;
    }
}
}

```

Таким образом, вся реализация криптографического алгоритма RSA, как и в случае с DES, представляет собой гибкое приложение с разными классами и возможностью их изменения, многочисленными тестами и удобным представлением в формате демо и полноценного приложения.

Rijndael

Реализация Rijndael алгоритма точно также вынесена в качестве отдельной работы со своей архитектурой, которая по содержанию в целом идентична представленным в листингах выше архитектурам для DES и DEAL. Тут главный класс уже сам содержит реализацию всех 4 функций, используемых вместе почти в каждом раунде.

Листинг 13. Класс RijndaelCipher.cs

```

using CryptoLib.Rijndael.Algorithms.Rijndael.GaloisField;
using CryptoLib.Rijndael.Algorithms.Rijndael.Enums;
using CryptoLib.Rijndael.Interfaces;
namespace CryptoLib.Rijndael.Algorithms.Rijndael
{

```

```

public class RijndaelCipher : ISymmetricCipher
{
    public int BlockSize { get; }
    public int KeySize { get; }
    private readonly int _rounds;
    private readonly SBox _sBox;
    private byte[][]? _roundKeys;

    public RijndaelCipher(KeySize keySizeEnum, BlockSize blockSizeEnum,
byte irreduciblePolynomial = 0x1B)
    {
        KeySize = (int)keySizeEnum / 8;
        BlockSize = (int)blockSizeEnum / 8;
        _rounds = GetNumberOfRounds(KeySize, BlockSize);
        _sBox = new SBox(irreduciblePolynomial);
    }

    public void SetRoundKeys(byte[] key)
    {
        if (key == null || key.Length != KeySize)
        {
            throw new ArgumentException($"Key length must be {KeySize}
bytes.");
        }

        var keyScheduler = new RijndaelKeyScheduler(_sBox);
        _roundKeys = keyScheduler.ExpandKey(key, BlockSize, KeySize,
_rounds);
    }

    public byte[] EncryptBlock(byte[] block)
    {
        if (_roundKeys == null)
            throw new InvalidOperationException("Round keys are not
set. Call SetRoundKeys() first.");

        if (block == null || block.Length != BlockSize)

```

```

        throw new ArgumentException($"Block must not be null and
size must be {BlockSize} bytes.");

        var state = ToStateMatrix(block);
        AddRoundKey(state, 0);
        for (int round = 1; round < _rounds; round++)
        {
            SubBytes(state);
            ShiftRows(state);
            MixColumns(state);
            AddRoundKey(state, round);
        }
        SubBytes(state);
        ShiftRows(state);
        AddRoundKey(state, _rounds);
        return FromStateMatrix(state);
    }

    public byte[] DecryptBlock(byte[] block)
    {
        if (_roundKeys == null)
            throw new InvalidOperationException("Round keys are not
set. Call SetRoundKeys() first.");
        if (block == null || block.Length != BlockSize)
            throw new ArgumentException($"Block must not be null and
size must be {BlockSize} bytes.");
        var state = ToStateMatrix(block);
        AddRoundKey(state, _rounds);
        InvShiftRows(state);
        InvSubBytes(state);
        for (int round = _rounds - 1; round >= 1; round--)
        {
            AddRoundKey(state, round);

```

```

        InvMixColumns(state);
        InvShiftRows(state);
        InvSubBytes(state);
    }
    AddRoundKey(state, 0);
    return FromStateMatrix(state);
}

private int GetNumberOfRounds(int keySizeBytes, int blockSizeBytes)
{
    int nk = keySizeBytes / 4;
    int nb = blockSizeBytes / 4;
    if (nk <= 6 && nb <= 6)
    {
        if (nk > 4 || nb > 4) return 12;
        return 10;
    }
    if (nk <= 8 && nb <= 8)
    {
        return 14;
    }

    throw new InvalidOperationException("Unsupported key/block size
combination for determining number of rounds.");
}

#region Private Helper Methods for Encryption
private byte[,] ToStateMatrix(byte[] block)
{
    int nb = BlockSize / 4; // Количество колонок
    var state = new byte[4, nb];
    for (int c = 0; c < nb; c++)
    {

```

```

        for (int r = 0; r < 4; r++)
        {
            state[r, c] = block[c * 4 + r];
        }
    }
    return state;
}

private byte[] FromStateMatrix(byte[,] state)
{
    int nb = BlockSize / 4;
    var block = new byte[BlockSize];
    for (int c = 0; c < nb; c++)
    {
        for (int r = 0; r < 4; r++)
        {
            block[c * 4 + r] = state[r, c];
        }
    }
    return block;
}

private void AddRoundKey(byte[,] state, int round)
{
    int nb = BlockSize / 4;
    for (int c = 0; c < nb; c++)
    {
        for (int r = 0; r < 4; r++)
        {
            state[r, c] ^= _roundKeys![round][c * 4 + r];
        }
    }
}

```

```

    }
    private void SubBytes(byte[,] state)
    {
        int nb = BlockSize / 4;
        for (int c = 0; c < nb; c++)
        {
            for (int r = 0; r < 4; r++)
            {
                state[r, c] = _sBox.SBoxTable[state[r, c]];
            }
        }
    }
    private void ShiftRows(byte[,] state)
    {
        int nb = BlockSize / 4;
        int[] shifts = nb switch
        {
            4 => [0, 1, 2, 3], // 128-bit block (AES)
            6 => [0, 1, 2, 3], // 192-bit block
            8 => [0, 1, 3, 4], // 256-bit block
            _ => throw new InvalidOperationException("Unsupported block
size.")
        };
        for (int r = 1; r < 4; r++)
        {
            var tempRow = new byte[nb];
            for (int c = 0; c < nb; c++)
            {
                tempRow[c] = state[r, c];
            }

```

```

        for (int c = 0; c < nb; c++)
        {
            state[r, c] = tempRow[(c + shifts[r]) % nb];
        }
    }
}

private void MixColumns(byte[,] state)
{
    int nb = BlockSize / 4;
    var tempCol = new byte[4];
    for (int c = 0; c < nb; c++)
    {
        for (int r = 0; r < 4; r++)
        {
            tempCol[r] = state[r, c];
        }

        state[0, c] = (byte)(GaloisFieldMath.Multiply(tempCol[0],
2, 0x1B) ^ GaloisFieldMath.Multiply(tempCol[1], 3, 0x1B) ^ tempCol[2] ^
tempCol[3]));

        state[1, c] = (byte)(tempCol[0] ^
GaloisFieldMath.Multiply(tempCol[1], 2, 0x1B) ^
GaloisFieldMath.Multiply(tempCol[2], 3, 0x1B) ^ tempCol[3]);

        state[2, c] = (byte)(tempCol[0] ^ tempCol[1] ^
GaloisFieldMath.Multiply(tempCol[2], 2, 0x1B) ^
GaloisFieldMath.Multiply(tempCol[3], 3, 0x1B));

        state[3, c] = (byte)(GaloisFieldMath.Multiply(tempCol[0],
3, 0x1B) ^ tempCol[1] ^ tempCol[2] ^ GaloisFieldMath.Multiply(tempCol[3],
2, 0x1B));
    }
}

}

#endregion

#region Private Helper Methods for Decryption

private void InvSubBytes(byte[,] state)

```



```

{
    int nb = BlockSize / 4;
    for (int c = 0; c < nb; c++)
    {
        for (int r = 0; r < 4; r++)
        {
            state[r, c] = _sBox.InverseSBoxTable[state[r, c]];
        }
    }
}

private void InvShiftRows(byte[,] state)
{
    int nb = BlockSize / 4;
    int[] shifts = nb switch
    {
        4 => [0, 1, 2, 3], // 128-bit block (AES)
        6 => [0, 1, 2, 3], // 192-bit block
        8 => [0, 1, 3, 4], // 256-bit block
        _ => throw new InvalidOperationException("Unsupported block
size.")
    };

    for (int r = 1; r < 4; r++) // Первая строка (r=0) не
сдвигается
    {
        var tempRow = new byte[nb];
        for (int c = 0; c < nb; c++)
        {
            tempRow[c] = state[r, c];
        }
        for (int c = 0; c < nb; c++)
        {

```

```

        state[r, c] = tempRow[(c + nb - shifts[r]) % nb];
    }
}

private void InvMixColumns(byte[,] state)
{
    int nb = BlockSize / 4;
    var tempCol = new byte[4];
    for (int c = 0; c < nb; c++)
    {
        for (int r = 0; r < 4; r++)
        {
            tempCol[r] = state[r, c];
        }

        state[0, c] = (byte)(GaloisFieldMath.Multiply(tempCol[0],
0x0E, 0x1B) ^ GaloisFieldMath.Multiply(tempCol[1], 0x0B, 0x1B) ^
GaloisFieldMath.Multiply(tempCol[2], 0x0D, 0x1B) ^
GaloisFieldMath.Multiply(tempCol[3], 0x09, 0x1B));

        state[1, c] = (byte)(GaloisFieldMath.Multiply(tempCol[0],
0x09, 0x1B) ^ GaloisFieldMath.Multiply(tempCol[1], 0x0E, 0x1B) ^
GaloisFieldMath.Multiply(tempCol[2], 0x0B, 0x1B) ^
GaloisFieldMath.Multiply(tempCol[3], 0x0D, 0x1B));

        state[2, c] = (byte)(GaloisFieldMath.Multiply(tempCol[0],
0x0D, 0x1B) ^ GaloisFieldMath.Multiply(tempCol[1], 0x09, 0x1B) ^
GaloisFieldMath.Multiply(tempCol[2], 0x0E, 0x1B) ^
GaloisFieldMath.Multiply(tempCol[3], 0x0B, 0x1B));

        state[3, c] = (byte)(GaloisFieldMath.Multiply(tempCol[0],
0x0B, 0x1B) ^ GaloisFieldMath.Multiply(tempCol[1], 0x0D, 0x1B) ^
GaloisFieldMath.Multiply(tempCol[2], 0x09, 0x1B) ^
GaloisFieldMath.Multiply(tempCol[3], 0x0E, 0x1B));
    }
}

#endregion
}
}

```

Вся работа с расширением ключа тут вынесена в отдельный класс. Там же находятся константы раундов Rcon, вспомогательные методы SubWord, RotWord и иные. Главная функция – ExpandKey, которая непосредственно и выполняет расширение ключа.

Листинг 14. Функция ExpandKey

```
public byte[][] ExpandKey(byte[] key, int blockSizeBytes, int keySizeBytes,
int rounds)
{
    int nk = keySizeBytes / 4;
    int nb = blockSizeBytes / 4;
    int expandedKeyWordsCount = nb * (rounds + 1);
    var w = new byte[expandedKeyWordsCount * 4];
    Array.Copy(key, w, keySizeBytes);
    for (int i = nk; i < expandedKeyWordsCount; i++)
    {
        var temp = new byte[4];
        Array.Copy(w, (i - 1) * 4, temp, 0, 4);
        if (i % nk == 0)
        {
            temp = SubWord(RotWord(temp));
            temp[0] ^= Rcon[i / nk];
        }
        else if (nk > 6 && i % nk == 4)
        {
            temp = SubWord(temp);
        }
        int prevWordIndex = (i - nk) * 4;
        int currentWordIndex = i * 4;
        w[currentWordIndex] = (byte)(w[prevWordIndex] ^
temp[0]);
        w[currentWordIndex + 1] = (byte)(w[prevWordIndex + 1] ^
temp[1]);
    }
}
```

```

        w[currentWordIndex + 2] = (byte)(w[prevWordIndex + 2] ^
temp[2]);
        w[currentWordIndex + 3] = (byte)(w[prevWordIndex + 3] ^
temp[3]);
    }
    return FormatRoundKeys(w, blockSizeBytes, rounds);
}

```

Процедура расширения ключа выполняется в соответствии со стандартом FIPS-197. Sbox генерируются в соответствующих функциях, как это и предусматривает Rijndael.

Отдельно стоит отметить реализацию математических операций над полем Галуа, которая является основной как Rijndael, так и AES. Она представлена в отдельном классе, причем выполнена как для полиномов, принадлежащих полю Галуа, то есть для байтов, так и для потенциально бесконечных полиномов BigInt для большей гибкости и универсальности. Часть класса для полиномов из Галуа представлена в листинге ниже.

Листинг 15. Часть класса GaloisFieldMath для полиномов Галуа

```

public static byte Add(byte a, byte b) => (byte)(a ^ b);
    public static byte Multiply(byte a, byte b, byte module)
    {
        byte result = 0;
        byte hi_bit_set;
        while (b > 0)
        {
            if ((b & 1) == 1)
                result ^= a;
            hi_bit_set = (byte)(a & 0x80);
            a <<= 1;
            if (hi_bit_set != 0)
                a ^= module;
            b >>= 1;
        }
    }

```

```

        return result;
    }
    public static byte Inverse(byte element, byte module)
    {
        if (element == 0) return 0;
        return Power(element, 254, module);
    }
    private static byte Power(byte baseValue, int exponent, byte
module)
    {
        byte result = 1;
        while (exponent > 0)
        {
            if ((exponent & 1) == 1)
                result = Multiply(result, baseValue, module);
            baseValue = Multiply(baseValue, baseValue, module);
            exponent >>= 1;
        }
        return result;
    }
    public static bool IsIrreducible(byte polynomialByte)
    {
        if (polynomialByte == 0) return false;
        int p = 0x100 | polynomialByte;
        if ((p & 1) == 0) return false;
        int[] irreducibleFactors = {
            0b11,          // x+1 (степень 1)
            0b111,         // x2+x+1 (степень 2)
            0b1011,        // x3+x+1 (степень 3)
            0b1101,        // x3+x2+1 (степень 3)
        }
    }

```

```

        0b10011,    //  $x^4+x+1$  (степень 4)
        0b11001,    //  $x^4+x^3+1$  (степень 4)
        0b11111     //  $x^4+x^3+x^2+x+1$  (степень 4)
    };

    foreach (var factor in irreducibleFactors)
    {
        int remainder = GetRemainder(p, factor);
        if (remainder == 0)
        {
            return false;
        }
    }
    return true;
}

public static List<byte> FindAllIrreduciblePolynomials()
{
    var polynomials = new List<byte>();
    for (int i = 0; i <= 255; i++)
    {
        if (IsIrreducible((byte)i)) polynomials.Add((byte)i);
    }
    return polynomials;
}

private static int GetRemainder(int dividend, int divisor)
{
    if (divisor == 0) throw new DivideByZeroException();
    int divisorDegree = GetPolynomialDegree(divisor);
    while (GetPolynomialDegree(dividend) >= divisorDegree)
    {

```

```

        int degreeDifference = GetPolynomialDegree(dividend) -
divisorDegree;

        dividend ^= divisor << degreeDifference;
    }
    return dividend;
}

private static int GetPolynomialDegree(int polynomial)
{
    for (int i = 31; i >= 0; i--)
    {
        if (((polynomial >> i) & 1) == 1) return i;
    }
    return -1;
}

```

Режимы набивки и шифрования не отличаются от уже представленных в листингах выше, а потом в данном случае соответствующие классы с незначительными изменениями просто скопированы в данную реализацию. В целом же Rijndael, подобно другим криптографическим алгоритмам, выполнен с той же гибкостью и универсальностью, тестируемостью и удобным представлением.

RC4

Реализация алгоритма RC4 за счет его простоты представляет собой всего 1 файл с соответствующим классом, который инкапсулируют все операции в собственных методах и не обращается к иным классам.

Листинг 16. Класс RC4Algorithm

```

using System;
using System.IO;
using System.Threading.Tasks;
namespace CryptoLib.New.Algorithms.RC4
{
    public class RC4Algorithm

```

```

{
    private byte[] _s; // State vector (S-Box)
    private int _i;
    private int _j;
    public RC4Algorithm(byte[] key)
    {
        if (key == null || key.Length == 0)
            throw new ArgumentException("Key must not be null or
empty.");
        InitializeState(key);
    }
    private void InitializeState(byte[] key)
    {
        _s = new byte[256];
        for (int i = 0; i < 256; i++)
        {
            _s[i] = (byte)i;
        }
        int j = 0;
        for (int i = 0; i < 256; i++)
        {
            j = (j + _s[i] + key[i % key.Length]) % 256;
            Swap(_s, i, j);
        }
        _i = 0;
        _j = 0;
    }
    public byte[] ProcessData(byte[] data)
    {
        byte[] result = new byte[data.Length];

```



```

        for (int k = 0; k < data.Length; k++)
        {
            _i = (_i + 1) % 256;
            _j = (_j + _s[_i]) % 256;
            Swap(_s, _i, _j);
            byte kByte = _s[( _s[_i] + _s[_j]) % 256];
            result[k] = (byte)(data[k] ^ kByte);
        }
        return result;
    }

    public async Task ProcessFileAsync(string inputFilePath, string
outputFilePath)
    {
        if (!File.Exists(inputFilePath))
            throw new FileNotFoundException($"File not found:
{inputFilePath}");

        const int bufferSize = 1024 * 64; // 64 KB
        byte[] buffer = new byte[bufferSize];

        using (var sourceStream = new FileStream(inputFilePath,
FileMode.Open, FileAccess.Read, FileShare.Read, bufferSize, true))
            using (var destinationStream = new FileStream(outputFilePath,
FileMode.Create, FileAccess.Write, FileShare.None, bufferSize, true))
            {
                int bytesRead;
                while ((bytesRead = await sourceStream.ReadAsync(buffer, 0,
buffer.Length)) > 0)
                {
                    byte[] processedChunk = new byte[bytesRead];
                    for (int k = 0; k < bytesRead; k++)
                    {
                        _i = (_i + 1) % 256;
                        _j = (_j + _s[_i]) % 256;

```

```

        Swap(_s, _i, _j);
        byte kByte = _s[(_s[_i] + _s[_j]) % 256];
        processedChunk[k] = (byte)(buffer[k] ^ kByte);
    }
    await destinationStream.WriteAsync(processedChunk, 0,
bytesRead);
    }
}

private void Swap(byte[] array, int index1, int index2)
{
    byte temp = array[index1];
    array[index1] = array[index2];
    array[index2] = temp;
}
}
}

```

Для реализации точно также написаны тесты и добавлена вкладка в общем приложении.

LOKI97

Данный криптоалгоритм, в отличие от предыдущего, уже представлен в 3 различных файлах из-за своей масштабности: файл с основным алгоритмом, файл для генерации S-Box-ов и вспомогательный класс, куда вынесены все константы и таблицы.

Листинг 17. Класс LOKI97Algorithm.cs

```

using System;
using CryptoLib.DES.Interfaces;
namespace CryptoLib.New.Algorithms.LOKI97
{

```

```

public class LOKI97Algorithm : ISymmetricCipher
{
    public int BlockSize => 16; // 128 бит
    public int KeySize { get; private set; }
    private const byte DEFAULT_POLYNOMIAL = 0x1B;
    private const int NUM_ROUNDS = 16;
    private const int NUM_SUBKEYS = 48;
    private static readonly byte[] P_TABLE =
    {
        56, 48, 40, 32, 24, 16, 8, 0,
        57, 49, 41, 33, 25, 17, 9, 1,
        58, 50, 42, 34, 26, 18, 10, 2,
        59, 51, 43, 35, 27, 19, 11, 3,
        60, 52, 44, 36, 28, 20, 12, 4,
        61, 53, 45, 37, 29, 21, 13, 5,
        62, 54, 46, 38, 30, 22, 14, 6,
        63, 55, 47, 39, 31, 23, 15, 7
    };
    private ulong[] _subkeys = null!;
    private byte[] _s1 = null!;
    private byte[] _s2 = null!;
    private bool _keysSet = false;
    private readonly byte _polynomial;
    public LOKI97Algorithm(byte polynomial = DEFAULT_POLYNOMIAL)
    {
        KeySize = 16;
        _polynomial = polynomial;
        InitializeSBoxes();
    }
}

```

```

        public LOKI97Algorithm(int keySizeInBytes, byte polynomial =
DEFAULT_POLYNOMIAL)
            : this(polynomial)
        {
            if (keySizeInBytes != 16 && keySizeInBytes != 24 &&
keySizeInBytes != 32)
                throw new ArgumentException("LOKI97 supports 16, 24, 32
bytes keys.");
            KeySize = keySizeInBytes;
        }
        private void InitializeSBoxes()
        {
            var tables = LOKI97SBoxGenerator.GenerateSBoxes(_polynomial);
            _s1 = tables.S1;
            _s2 = tables.S2;
        }
        public void SetRoundKeys(byte[] key)
        {
            if (key == null) throw new ArgumentNullException(nameof(key));
            if (key.Length != 16 && key.Length != 24 && key.Length != 32)
                throw new ArgumentException("Invalid key size.");
            KeySize = key.Length;
            _subkeys = GenerateSubkeys(key);
            _keysSet = true;
        }
        public byte[] EncryptBlock(byte[] block)
        {
            if (!_keysSet) throw new InvalidOperationException("Keys not
set");
            if (block.Length != 16) throw new ArgumentException("Block size
must be 16 bytes");

```

```

        ulong L = BytesToUlong(block, 0);
        ulong R = BytesToUlong(block, 8);
        for (int i = 0; i < NUM_ROUNDS; i++)
        {
            ulong keyA = _subkeys[3 * i];
            ulong keyB = _subkeys[3 * i + 1];
            ulong keyC = _subkeys[3 * i + 2];
            ulong sum = unchecked(R + keyA);
            ulong f_out = F(sum, keyB);
            ulong newR = L ^ f_out;
            ulong newL = unchecked(R + keyC);
            L = newL;
            R = newR;
        }
        byte[] output = new byte[16];
        UlongToBytes(R, output, 0);
        UlongToBytes(L, output, 8);
        return output;
    }

    public byte[] DecryptBlock(byte[] block)
    {
        if (!_keysSet) throw new InvalidOperationException("Keys not
set");

        if (block.Length != 16) throw new ArgumentException("Block size
must be 16 bytes");

        ulong R = BytesToUlong(block, 0);
        ulong L = BytesToUlong(block, 8);
        for (int i = NUM_ROUNDS - 1; i >= 0; i--)
        {
            ulong keyA = _subkeys[3 * i];
            ulong keyB = _subkeys[3 * i + 1];

```

```

        ulong keyC = _subkeys[3 * i + 2];
        ulong prevR = unchecked(L - keyC);
        ulong f_out = F(unchecked(prevR + keyA), keyB);
        ulong prevL = R ^ f_out;
        L = prevL;
        R = prevR;
    }
    byte[] output = new byte[16];
    UlongToBytes(L, output, 0);
    UlongToBytes(R, output, 8);
    return output;
}

private ulong F(ulong A, ulong B)
{
    ulong state = A ^ B;
    ulong sbbox_output = 0;
    byte[] sbbox_pattern = { 1, 2, 1, 2, 2, 1, 2, 1 };
    for (int i = 0; i < 8; i++)
    {
        int shift = (7 - i) * 8;
        byte val = (byte)((state >> shift) & 0xFF);
        byte s_out;
        if (sbbox_pattern[i] == 1) s_out = _s1[val];
        else s_out = _s2[val];
        sbbox_output |= ((ulong)s_out << shift);
    }
    return Permute64(sbbox_output);
}

private ulong Permute64(ulong input)
{

```

```

        ulong output = 0;
        for (int i = 0; i < 64; i++)
        {
            if (((input >> i) & 1) == 1)
            {
                int targetPos = P_TABLE[i];
                output |= (1UL << targetPos);
            }
        }
        return output;
    }

    private ulong[] GenerateSubkeys(byte[] key)
    {
        int numWords = KeySize / 8;
        ulong[] K = new ulong[NUM_SUBKEYS];
        for (int i = 0; i < NUM_SUBKEYS; i++)
        {
            int byteIndex = (i * 8) % key.Length;
            ulong kVal = 0;
            for(int j=0; j<8; j++)
            {
                kVal = (kVal << 8) | key[(byteIndex + j) % key.Length];
            }
            K[i] = F(kVal ^ delta, kVal);
        }
        return K;
    }

    private ulong BytesToUlong(byte[] b, int offset)
    {
        ulong res = 0;

```

```

        for (int i = 0; i < 8; i++) res = (res << 8) | b[offset + i];
        return res;
    }
    private void UlongToBytes(ulong u, byte[] b, int offset)
    {
        for (int i = 0; i < 8; i++) b[offset + i] = (byte)((u >> (7 -
i) * 8) & 0xFF);
    }
}
}

```

Как видно из представленного листинга, подобно RC4, данный класс также инкапсулирует всю логику внутри себя, делегируя иному классу лишь создание S-Box.

Листинг 18. Класс LOKI97SBoxGenerator.cs

```

using System;

namespace CryptoLib.New.Algorithms.LOKI97
{
    internal static class LOKI97SBoxGenerator
    {
        public static (byte[] S1, byte[] S2) GenerateSBoxes(byte
polynomial)
        {
            byte[] s1 = new byte[256];
            byte[] s2 = new byte[256];
            int fullPoly = 0x100 | polynomial;
            for (int i = 0; i < 256; i++)
            {
                byte x = (byte)i;
                byte x2 = GF_Multiply(x, x, fullPoly); // x^2
                byte x3 = GF_Multiply(x2, x, fullPoly); // x^3
                s1[i] = x3;
            }
        }
    }
}

```



```

        s2[i] = GF_Inverse(x, fullPoly);
    }
    return (s1, s2);
}

private static byte GF_Multiply(byte a, byte b, int poly)
{
    int p = 0;
    int aa = a;
    int bb = b;
    for (int i = 0; i < 8; i++)
    {
        if ((bb & 1) != 0)
        {
            p ^= aa;
        }
        bool highBitSet = (aa & 0x80) != 0;
        aa <<= 1;
        if (highBitSet)
        {
            aa ^= poly; // XOR с полиномом
        }
        bb >>= 1;
    }
    return (byte)p;
}

private static byte GF_Inverse(byte b, int poly)
{
    if (b == 0) return 0;
    byte result = 1;
    byte baseVal = b;

```

```

        int exponent = 254;
        while (exponent > 0)
        {
            if ((exponent & 1) != 0)
            {
                result = GF_Multiply(result, baseVal, poly);
            }
            baseVal = GF_Multiply(baseVal, baseVal, poly); // square
            exponent >>= 1;
        }
        return result;
    }
}
}

```

Для алгоритма также скопирован класс CipherContext с тем лишь изменением, что в нем отсутствуют выбор иных алгоритмов. Аналогично написаны тесты для проверки работоспособности всей функциональности, включая все режимы набивки, форматы файлов и сравнение по скорости. Наконец, также реализовано и приложение.

Diffie-Hellman

Вся реализация протокола представлена в формате 1 файла, где используется весь нужный математический аппарат для возможности через незащищенный канал в конечном итоге создавать одинаковый секретный ключ.

Листинг 19. Класс DiffieHellmanProtocol.cs

```

using System;
using System.Linq;
using System.Numerics;
using System.Security.Cryptography;
namespace CryptoLib.New.Protocols.DiffieHellman
{

```

```

public class DiffieHellmanProtocol
{
    private static readonly BigInteger P_RFC3526 = BigInteger.Parse(
"32317006071311007300338913926423828248817941241140239112842009751400741706
634354222619689417363569347117901737909704191754605873209195028853758986185
622153212175412514901774520270235796078236248884246189479356346697242213174
620506797232575204446048677422131067303602122718715856683348665348562605191
4642966629"

    );

    private static readonly BigInteger G_RFC3526 = 2;

    public BigInteger P { get; }
    public BigInteger G { get; }
    private BigInteger _privateKey;
    public BigInteger PublicKey { get; private set; }
    public DiffieHellmanProtocol()
    {
        P = P_RFC3526;
        G = G_RFC3526;
        GenerateKeyPair();
    }
    public DiffieHellmanProtocol(BigInteger p, BigInteger g)
    {
        P = p;
        G = g;
        GenerateKeyPair();
    }
    private void GenerateKeyPair()
    {
        _privateKey = GenerateRandomSecret(32);
        PublicKey = BigInteger.ModPow(G, _privateKey, P);
    }
    public BigInteger CalculateSharedSecret(BigInteger otherPublicKey)

```

```

    {
        return BigInteger.ModPow(otherPublicKey, _privateKey, P);
    }

    public static byte[] DeriveSymmetricKey(BigInteger sharedSecret,
int keySizeInBytes)
    {
        byte[] secretBytes = sharedSecret.ToByteArray();
        using (var sha256 = SHA256.Create())
        {
            byte[] hash = sha256.ComputeHash(secretBytes);
            if (hash.Length >= keySizeInBytes)
            {
                byte[] key = new byte[keySizeInBytes];
                Array.Copy(hash, key, keySizeInBytes);
                return key;
            }
            else
            {
                throw new ArgumentException("Requested key size is too
large for SHA256 derivation");
            }
        }
    }

    private BigInteger GenerateRandomSecret(int length)
    {
        byte[] bytes = new byte[length];
        RandomNumberGenerator.Fill(bytes);
        bytes[bytes.Length - 1] &= 0x7F;
        if (bytes.All(b => b == 0)) bytes[0] = 1;
        return new BigInteger(bytes);
    }

```

}

}

Наглядное представление работоспособности алгоритма, помимо исчерпывающих тестов, представлено в формате окна общего приложения, где есть возможность задать нужные параметры, а после генерировать сообщение, шифровать его, отправлять и дешифровать по итогу.

Вывод

В процессе разработки была создана модульная библиотека криптографических алгоритмов, сочетающая в себе ключевые компоненты, востребованные в современных системах защиты информации. В неё вошли: симметричные блочные шифры DES, TripleDES, DEAL и Rijndael (AES) с поддержкой различных режимов работы, поточный шифр RC4, асимметричный алгоритм RSA и протокол обмена ключами Диффи-Хеллмана. Реализация охватывает как классические криптографические примитивы, так и современные стандарты шифрования.

Каждый модуль библиотеки ориентирован на корректность реализации криптографических преобразований и строгое следование спецификациям: применены сети Фейстеля для DES и его производных, структура SPN для Rijndael, арифметика в полях Галуа и модульная арифметика для RSA. Дополнительно проработаны механизмы расширения ключей, различные режимы работы блочных шифров (ECB, CBC, OFB, CFB, CTR, PCBC, RandomDelta) и тесты на простоту для генерации RSA-ключей.

Практическая значимость проекта заключается в его применимости для изучения внутреннего устройства криптографических алгоритмов: от анализа раундовых преобразований до понимания математических основ стойкости. Библиотека демонстрирует современные подходы к проектированию криптографического ПО и может служить как основой для интеграции отдельных компонентов в учебные проекты, так и наглядным примером для изучения принципов симметричного и асимметричного шифрования на практике.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. FIPS PUB 46-3. Data Encryption Standard (DES) / National Institute of Standards and Technology. – Gaithersburg, MD : NIST, 1999. – 22 p.
2. FIPS PUB 197. Advanced Encryption Standard (AES) / National Institute of Standards and Technology. – Gaithersburg, MD : NIST, 2001. – 51 p.
3. Schneier, B. Applied Cryptography: Protocols, Algorithms and Source Code in C / B. Schneier. – 2nd ed. – New York : John Wiley & Sons, 1996. – 784 p. – ISBN 978-0-471-11709-4.
4. Diffie, W. New Directions in Cryptography / W. Diffie, M. E. Hellman // IEEE Transactions on Information Theory. – 1976. – Vol. 22, № 6. – P. 644–654. – DOI: 10.1109/TIT.1976.1055638.
5. Rivest, R. L. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems / R. L. Rivest, A. Shamir, L. Adleman // Communications of the ACM. – 1978. – Vol. 21, № 2. – P. 120–126. – DOI: 10.1145/359340.359342.
6. NIST Special Publication 800-67. Recommendation for the Triple Data Encryption Algorithm (TDEA) Block Cipher / National Institute of Standards and Technology. – Revision 2. – Gaithersburg, MD : NIST, 2017. – 30 p.

Приложение А Репозиторий с исходным кодом

Cryptography_CW [Электронный ресурс]. // github —
URL: https://github.com/DmitriyKolesnikM8O/Cryptography_CW (дата обращения:
14.12.2025).